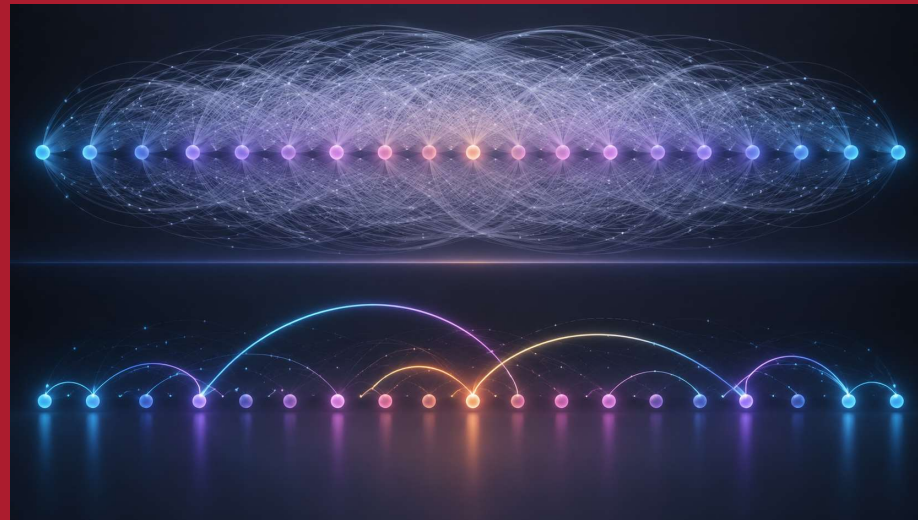


CS5326 – Advanced GenAI and Agents
Fall 2026

Agha Ali Raza

Efficient Attention



Readings

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 5998 to 6008.
<https://arxiv.org/abs/1706.03762>
- Liu, P. J., Saleh, M., Pot, E., Goodrich, B., Sepassi, R., Kaiser, Ł., and Shazeer, N. (2018). Generating Wikipedia by Summarizing Long Sequences. In *Proceedings of the 6th International Conference on Learning Representations (ICLR 2018)*.
<https://arxiv.org/abs/1801.10198>
- Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv preprint.
<https://arxiv.org/abs/1911.02150>
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pp. 4895 to 4901.
<https://arxiv.org/abs/2305.13245>
- Milakov, M., and Gimelshein, N. (2018). Online Normalizer Calculation for Softmax. arXiv preprint.
<https://arxiv.org/abs/1805.02867>
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.
<https://arxiv.org/abs/2205.14135>

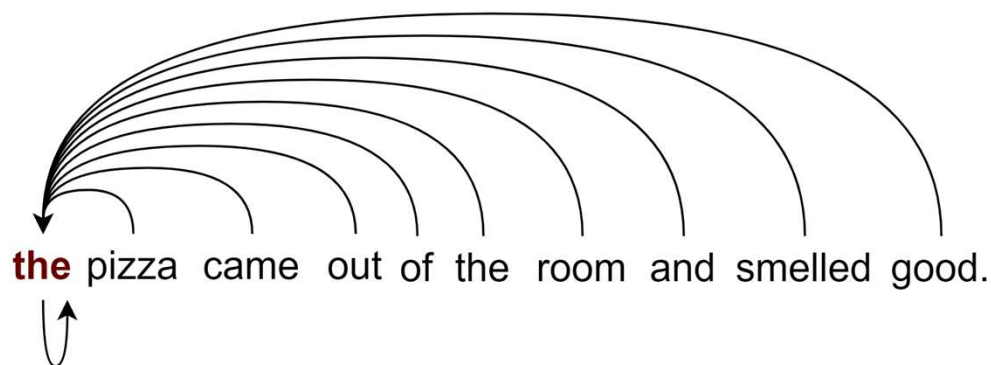
Revision: Let's Encode!

A Need for Context

- Take the sentence, “The chicken didn’t cross the street because *it* was too tired”
 - What does *it* in this sentence refer to? The **chicken** or the **street**?
 - I hate *tired streets* by the way.
- Associating an individual word with others is something very natural for humans, but difficult for machines.
 - **Self-Attention** is the mechanism with which Transformers can look at other positions in the input sequence for clues that can help lead to a better encoding for a specific word.
- RNNs use their hidden state to incorporate representation of previous words into the one it is currently processing.
 - **Self-Attention** is what Transformers use.

Self-Attention to **encode** the input sentence

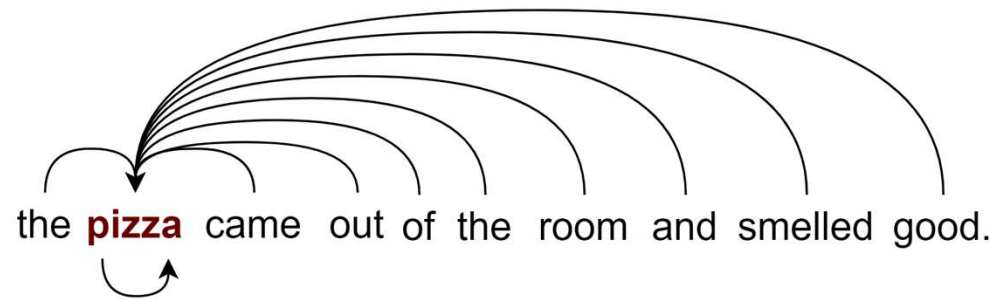
- For all of the words
 - In parallel
 - To form the embeddings for “the”



1. Find the dot product between “the” and every other word (including itself)
2. Softmax to find the attention weights that sum to 1
3. Multiply each word’s vector with its proportion to get scaled vectors
4. Add these together to form the identity of “the”

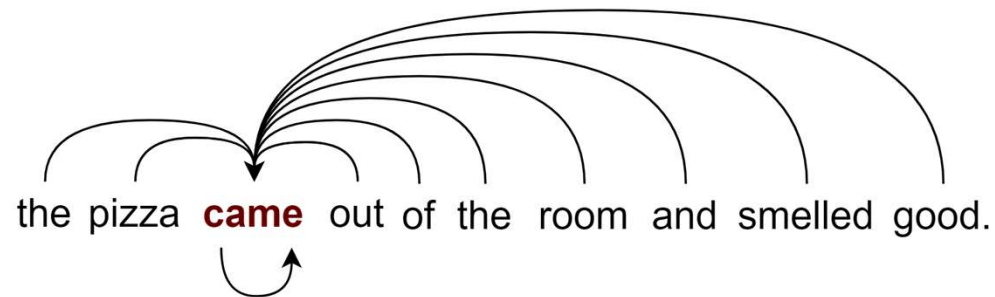
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



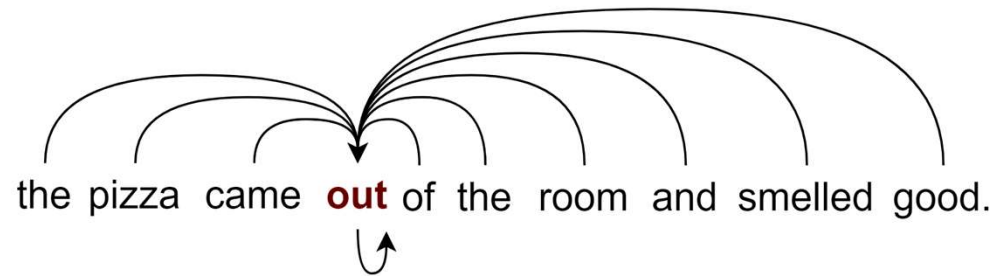
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



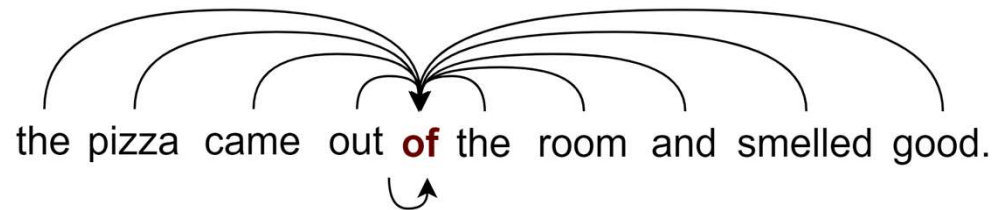
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



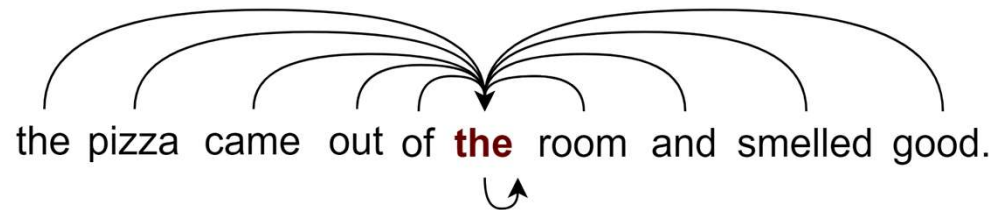
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



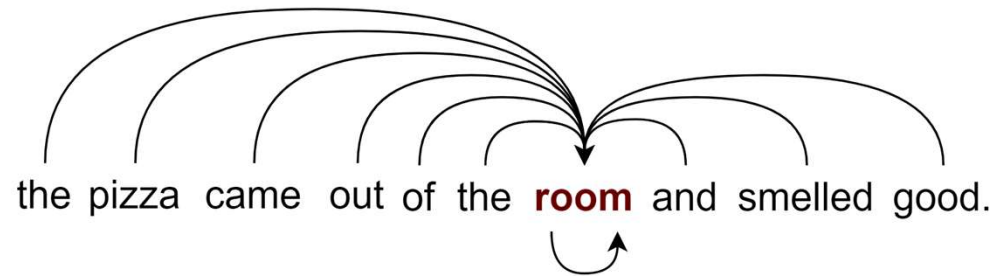
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



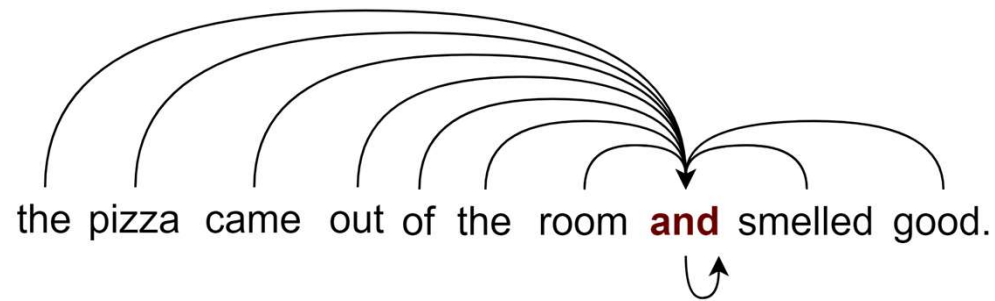
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



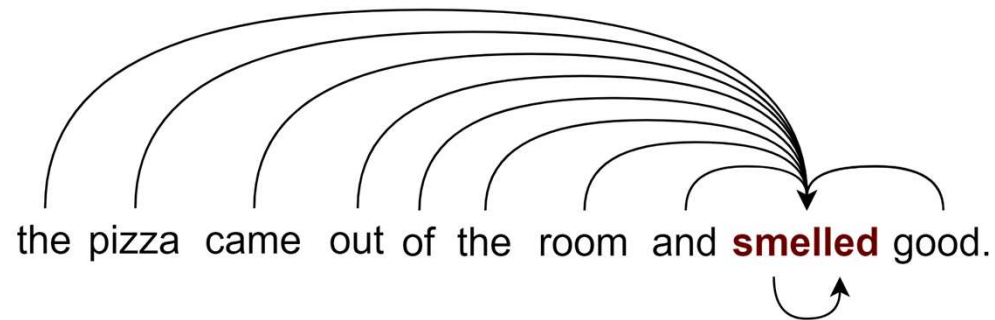
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



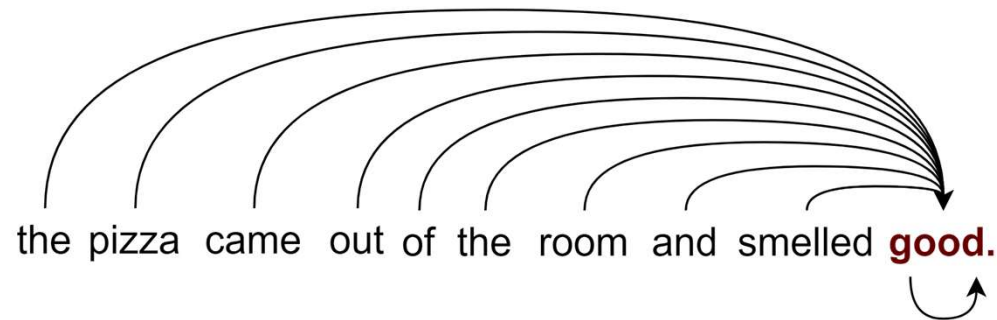
Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



Self-Attention to **encode** the input sentence

- For all of the words
 - In parallel



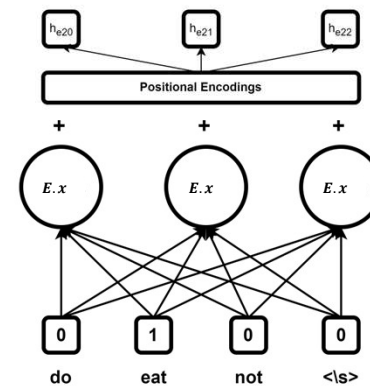
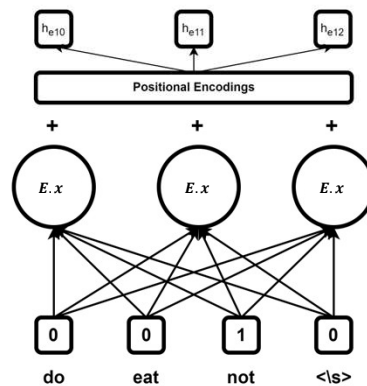
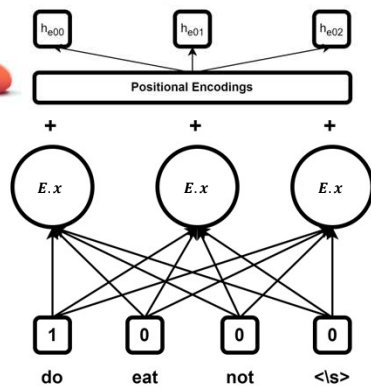
Time to match up!

Oh no! I look terrible!
What should I wear?

I wonder if we could
learn it!!!

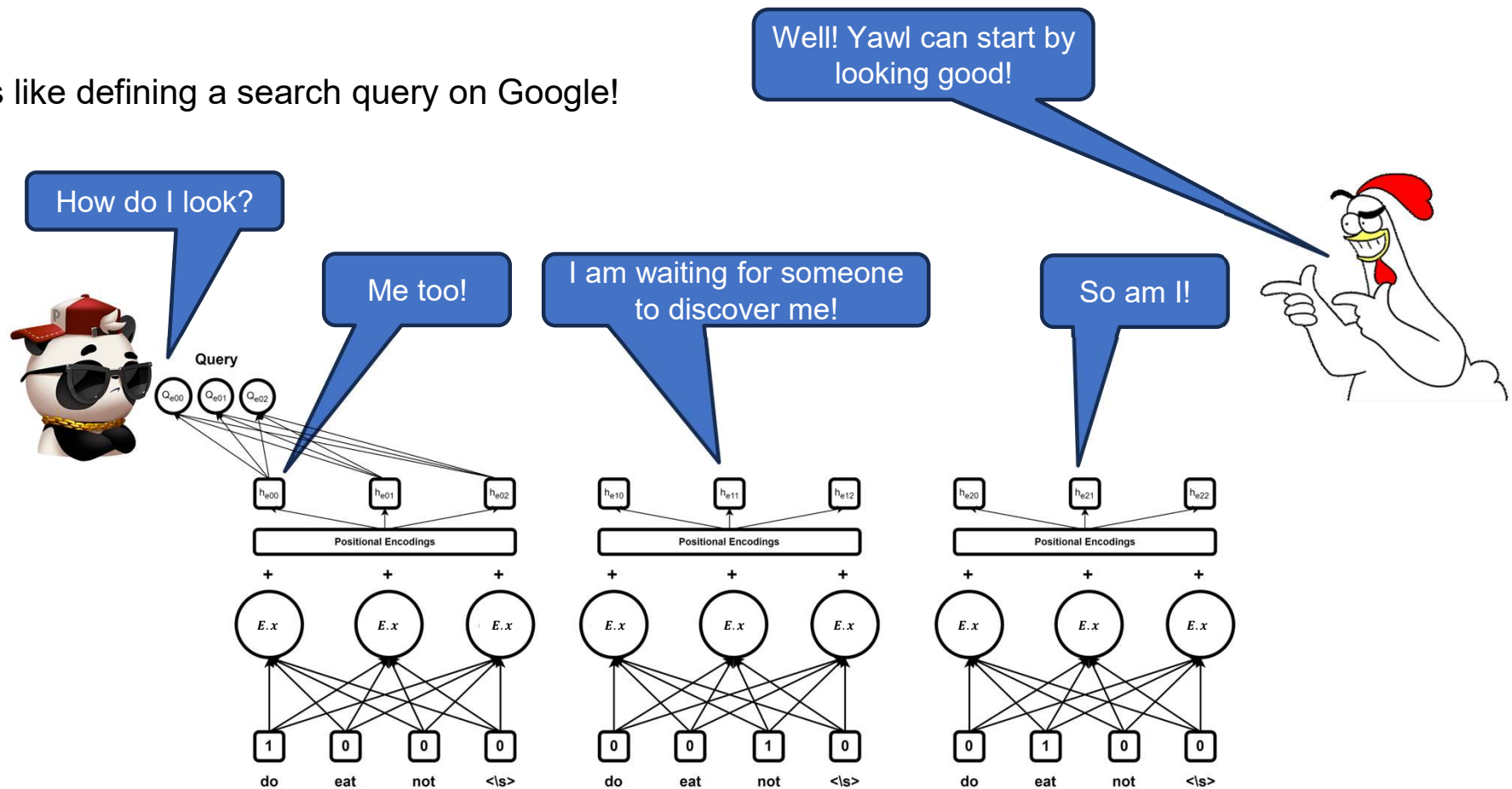
And *that's* what you
are wearing?

I'm looking for a
match!



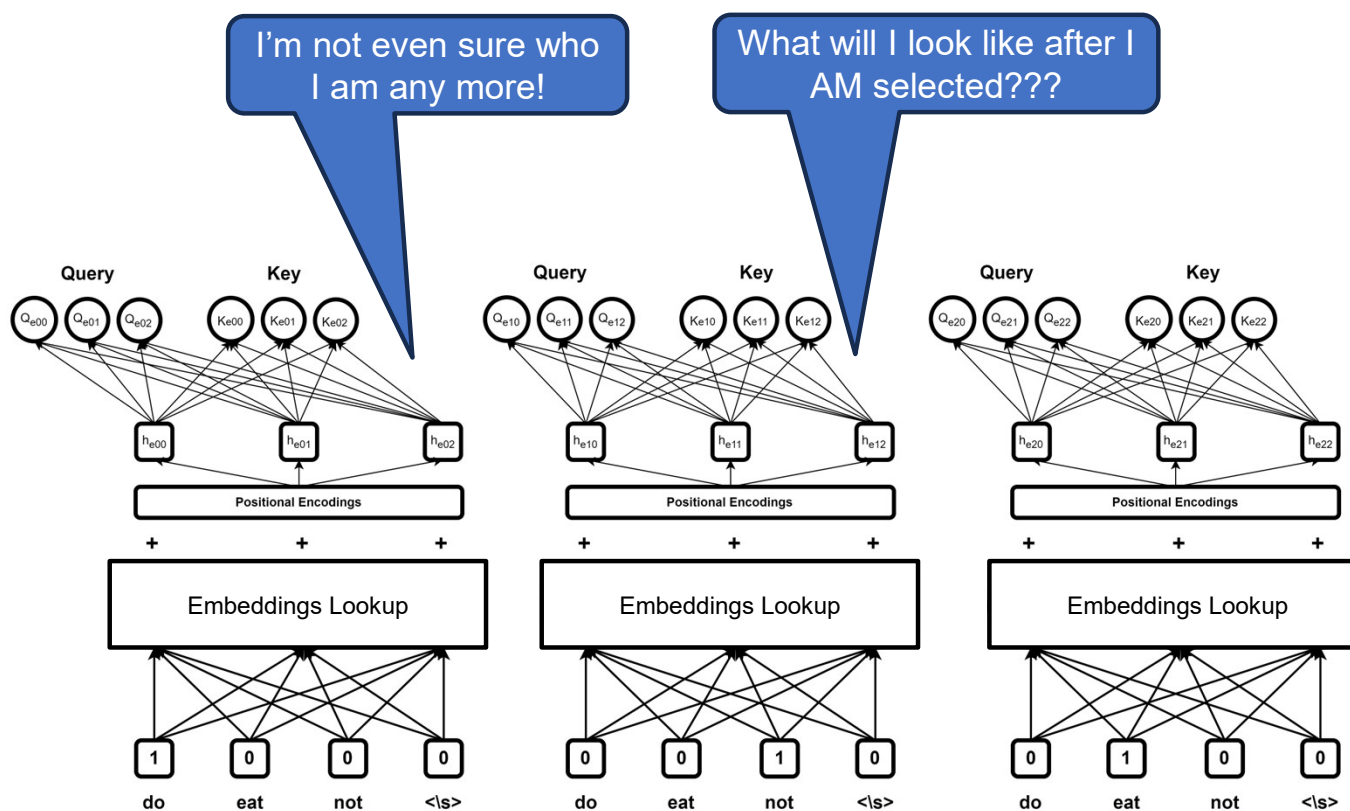
Time to match up!

This is like defining a search query on Google!



Time to match up!

This is like defining **keywords** on a page to be searched by Google!

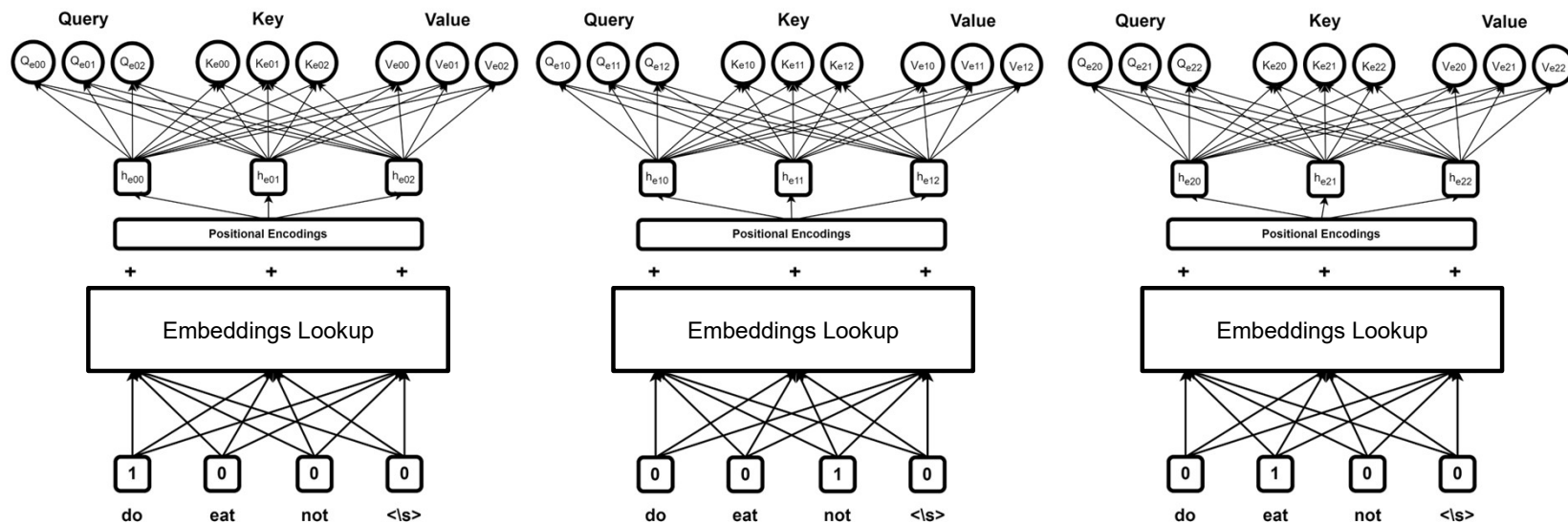


Time to match up!

Wow. Yawl look so nice
and complicated!!!

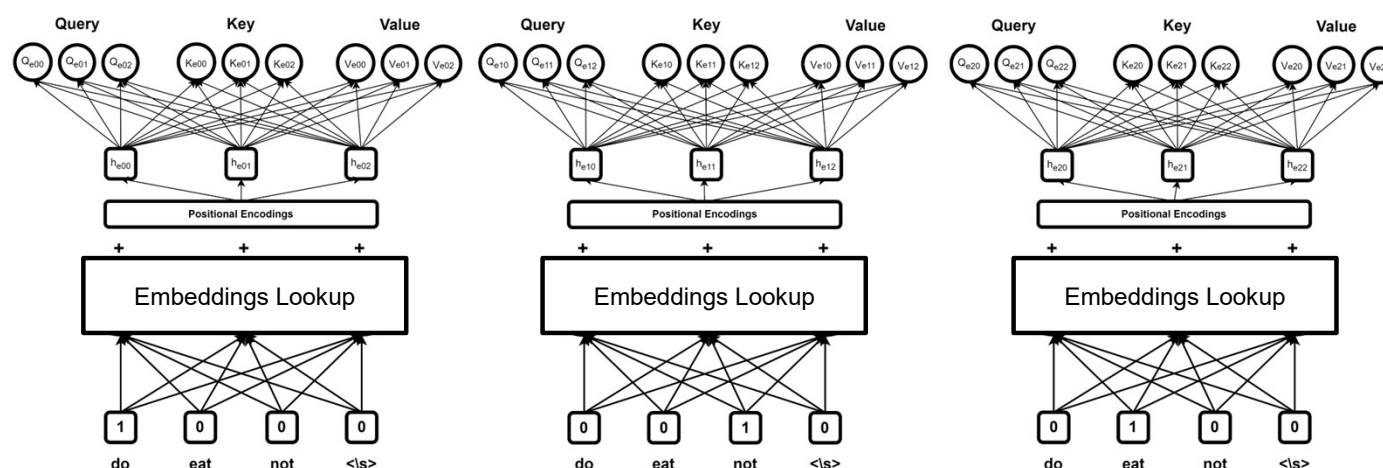


This is like the value (page) to be returned by Google!



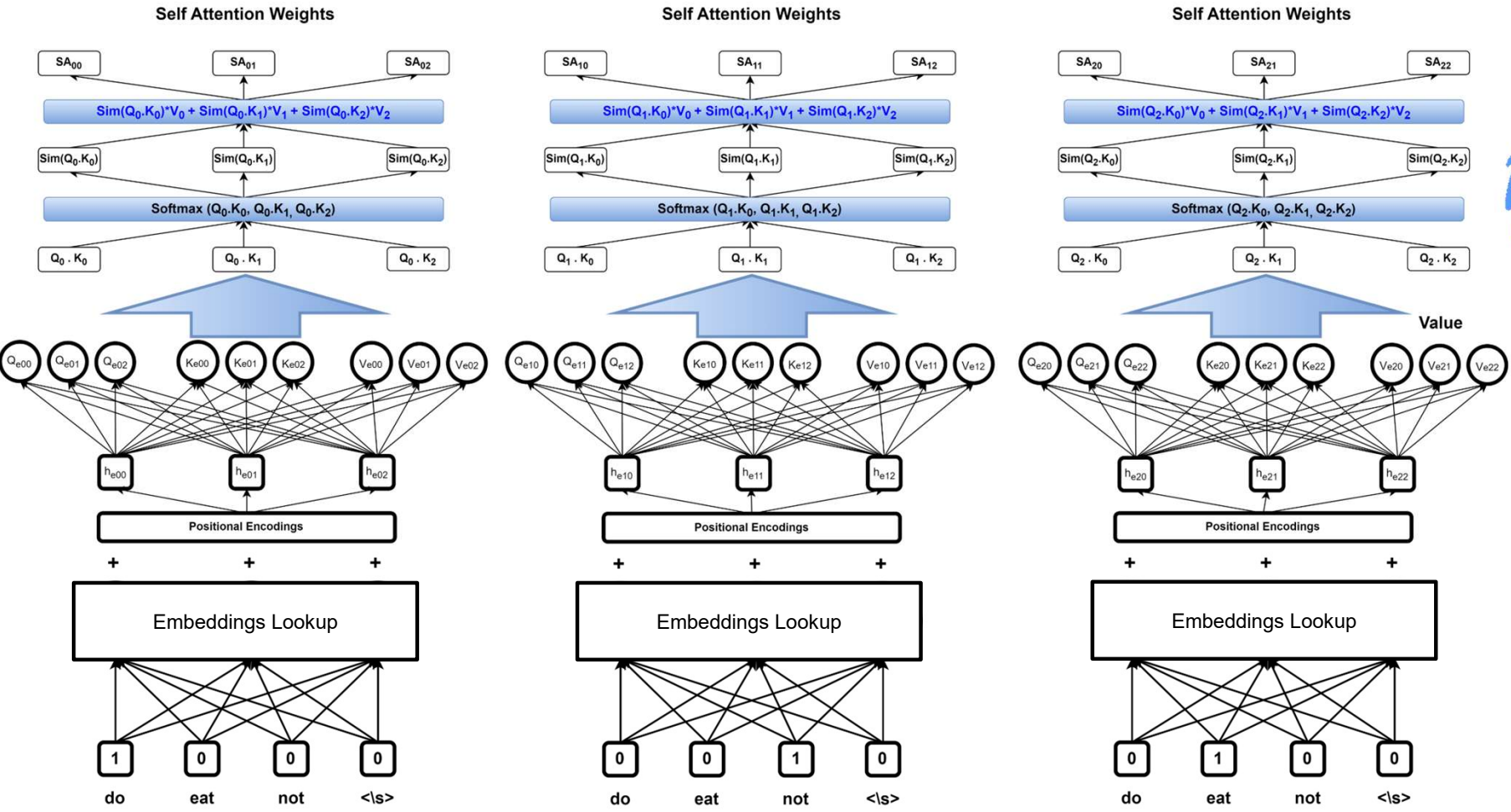
Time to match up!

- Within one head of one layer, the same W^Q, W^K, W^V apply at every token position. Different heads and layers normally have their own parameters.
- The Query vector for the word will also be matched with its own Key to measure the self similarity.
- The next few steps are very similar to what we have seen for the encoder-decoder attention:
 1. Find similarities using a **scaled dot product** – Between the query of each target words and all keys to get a **score** for each key.
 2. Scale the scores for each target word using the **Soft Max** function – which preserves order and scales the weights so that they sum to 1.
 3. Now, scale each **value** using its soft max contribution and add these all up to form the **Self Attention** vector.
- **The whole process happens in parallel for the entire encoder!**



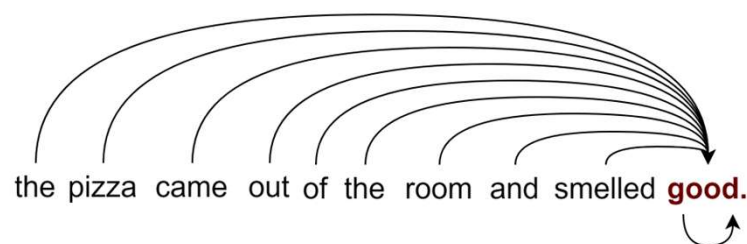
Checkpoint: Attention Weights

These are not tears! I am sooo happy!



The bigger picture

- These self-attention weights establish the relationships among words in a sentence.
- A set of query, key, and value matrices is called an **attention head**.



- For a complex paragraph or article where multiple layers of relationships exist among words, it makes sense to replicate these **self-attention cells** or **attention heads**
 - To capture different relationships among the words
- This is known as **multi-head attention**
- In the first paper on transformers 8 attention heads were used

Why 8 Heads? The Dimension Split

The Split Strategy

In "Attention Is All You Need":

```
d_model = 512  
num_heads = 8  
d_k = 512 / 8 = 64
```

Each head works with 64 dimensions instead of the full 512.

It's a trade-off!

The Trade-off

Fewer heads (e.g. 2)

Each head = 256 dims → richer per-head representation, but fewer patterns captured

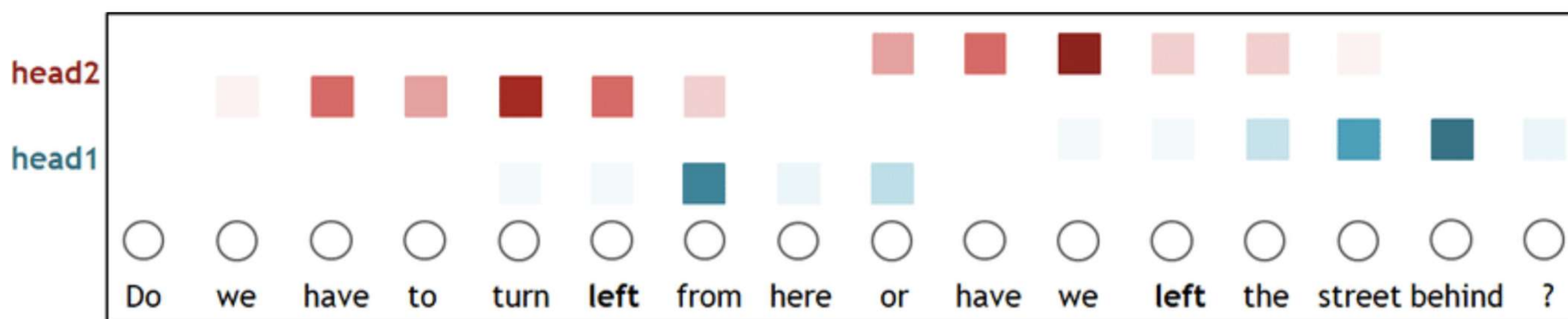
vs

More heads (e.g. 16)

Each head = 32 dims → more patterns captured, but less capacity per head

8 heads × 64 dims was the sweet spot found empirically by Vaswani et al. (2017)

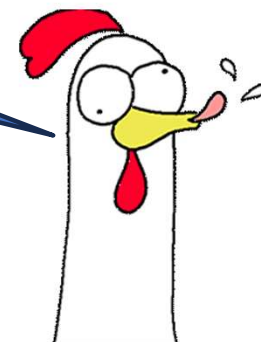
Multiple Attention Heads to deal with Multiple Parses



Ready to go crazy?

- Time flies like an arrow.
- Old men and women.
- I shot an elephant in my pajamas.
- Locked in a vault for fifty years, the owner decided to sell the jewels.
- Plunging a thousand feet into the gorge, we saw Yosemite falls.

Yes! Crazy is my middle name!



What Makes Each Head Different?

Each head has its own independently learned W_Q , W_K , W_V weight matrices — this is what makes them attend to different things.

Head A

Syntactic Relations

Subject $\leftrightarrow$ Verb agreement
"The cats ... are"

Head B

Positional / Local

Adjacent word patterns
"New York", "ice cream"

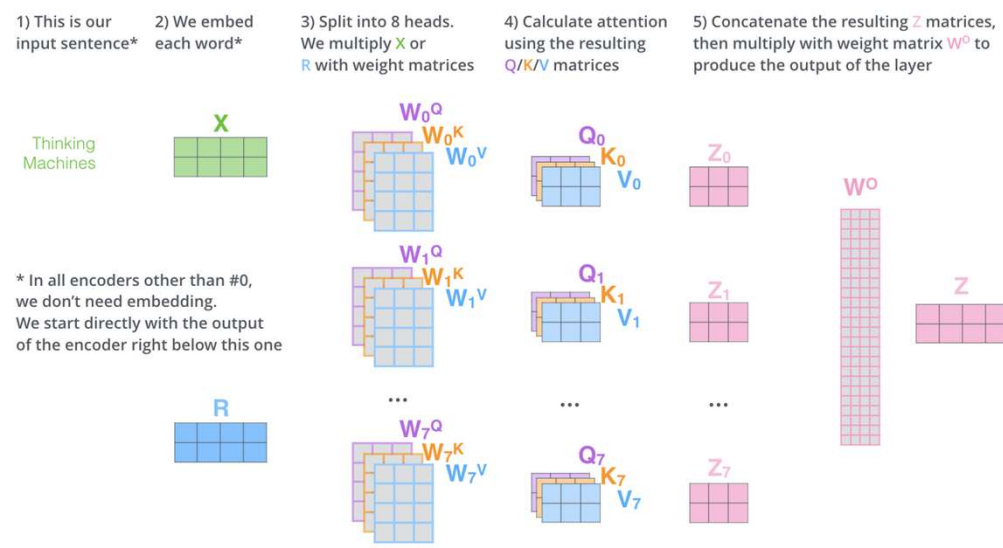
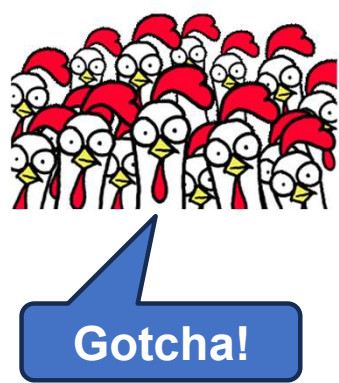
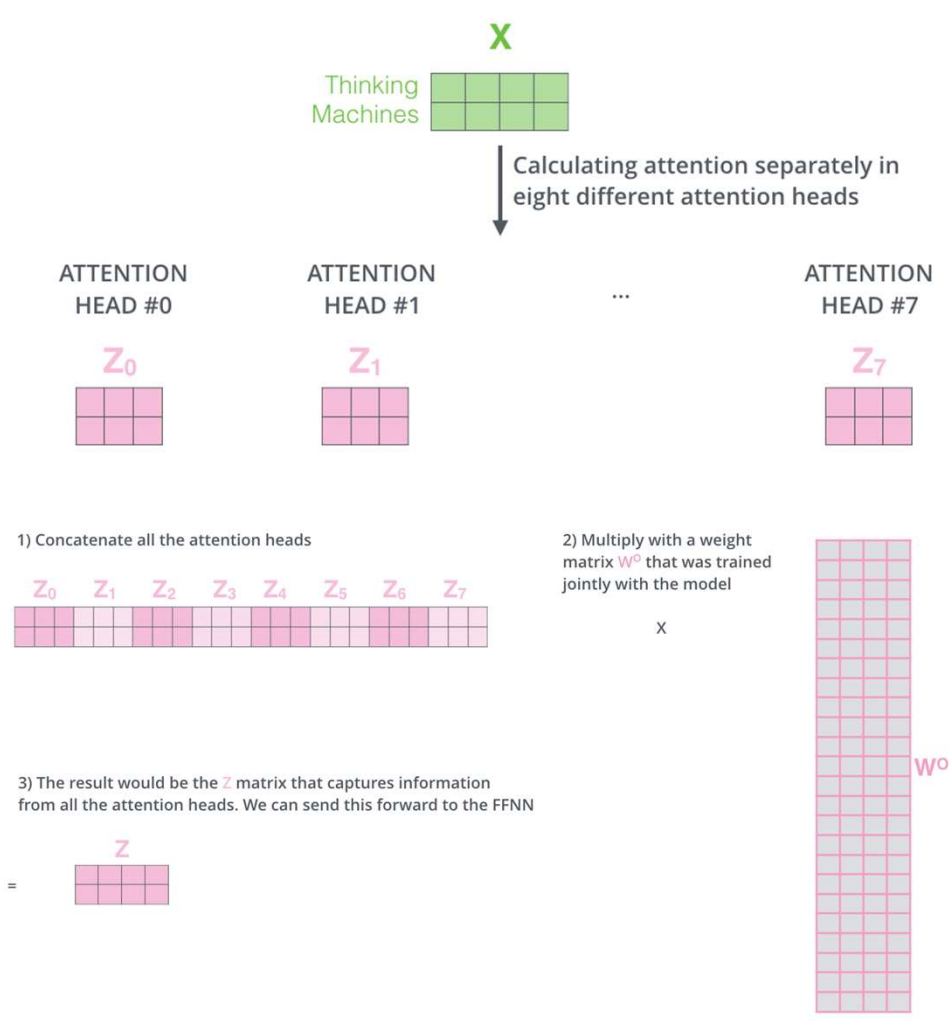
Head C

Semantic Links

Meaning connections
"doctor ... hospital"

*These specializations **may emerge** automatically during training — they are not manually designed.*

Multi-head Attention



Why Concatenate & Project with W^0 ?

$Z_0, Z_1 \dots Z_7$

8 head outputs
(each 64-dim)



Concatenate

Stack into one
512-dim vector



$\times W^0$

Learned linear
projection



Z (512-dim)

Same shape as
input X



Why does this matter?

- 1. Learned combination:** W^0 lets the model learn HOW to combine head outputs, not just concatenate them naively.
- 2. Dimension preservation:** Output Z has the same dimension (512) as input X . This enables residual connections and stacking layers.
- 3. Cross-head interaction:** The projection mixes information ACROSS heads, so the model can synthesize their findings.

Why not eight full-width heads?

Suppose each of the $h = 8$ heads projected to 512 dimensions instead of $d = 64$.

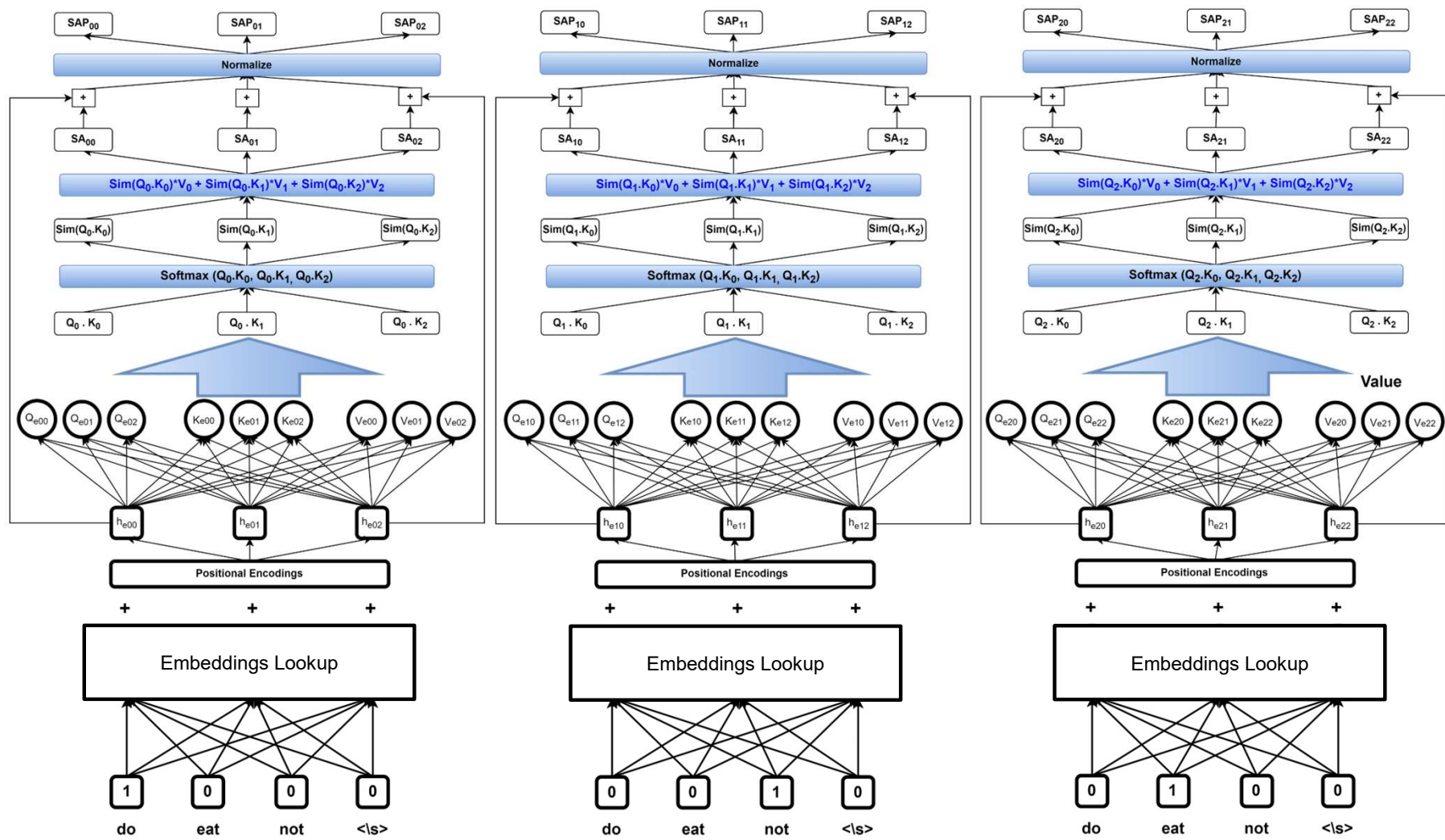
- **It costs 8× more.** Each W_Q, W_K, W_V becomes 512×512 , the score computation $n^2 d_k$ grows 8×, and the concatenation has width $8 \times 512 = 4096$, so W_O becomes 4096×512 . Eight narrow heads cost the same as one wide head.
- **It is more expressive, not less.** Every 64-wide head is a rank-64 special case of a 512-wide head. Narrowness does not create specialization; heads differ because of initialization and training, and they become partly redundant even at 64 dimensions (Voita et al., 2019; Michel et al., 2019).
- **So the choice is a budget decision.** Vaswani et al. (Table 3, row A) varied $h \in \{1, 4, 8, 16, 32\}$ with $d = 512/h$, holding total width and compute fixed. Eight heads beat one head by about 0.9 BLEU and slightly beat 16 and 32. At a fixed budget, several narrow perspectives beat one wide one. Full-width heads were never tested, and row B shows that shrinking d_k below 64 hurts, so per-head capacity still matters.
- **$hd = D$ is a convention, not a rule.** W_O maps $h \cdot d_v \rightarrow D$ for any h and d_v ; d_k need not equal d_v . Later work (Bhojanapalli et al., 2020) widens d_k independently and finds gains at long sequence lengths.

The Residual Connections from Positional Embeddings

We are big, fat numbers now. About to overflow. Normalize us!

Hey! Remind me what my position was!

Also, deal with the vanishing gradients



Residual Connection + LayerNorm

$$\text{Output} = \text{LayerNorm}(X + \text{MultiHeadAttention}(X))$$

Residual Connection (+ X)

The original input X is ADDED to the attention output (the "skip connection").

Why?

- Prevents vanishing gradients in deep networks (6+ layers)
- Lets each layer learn a DELTA (what to add), not a full transform

LayerNorm

Normalizes each token's vector to zero mean and unit variance.

Why?

- Stabilizes training — prevents activations from exploding/collapsing
- Enables higher learning rates
- Applied per-token, not per-batch

Without these two components, transformers cannot be stacked to 12, 24, or 96 layers deep.

Layer Norm

- Layer normalization stabilizes training by normalizing across features for each token independently:

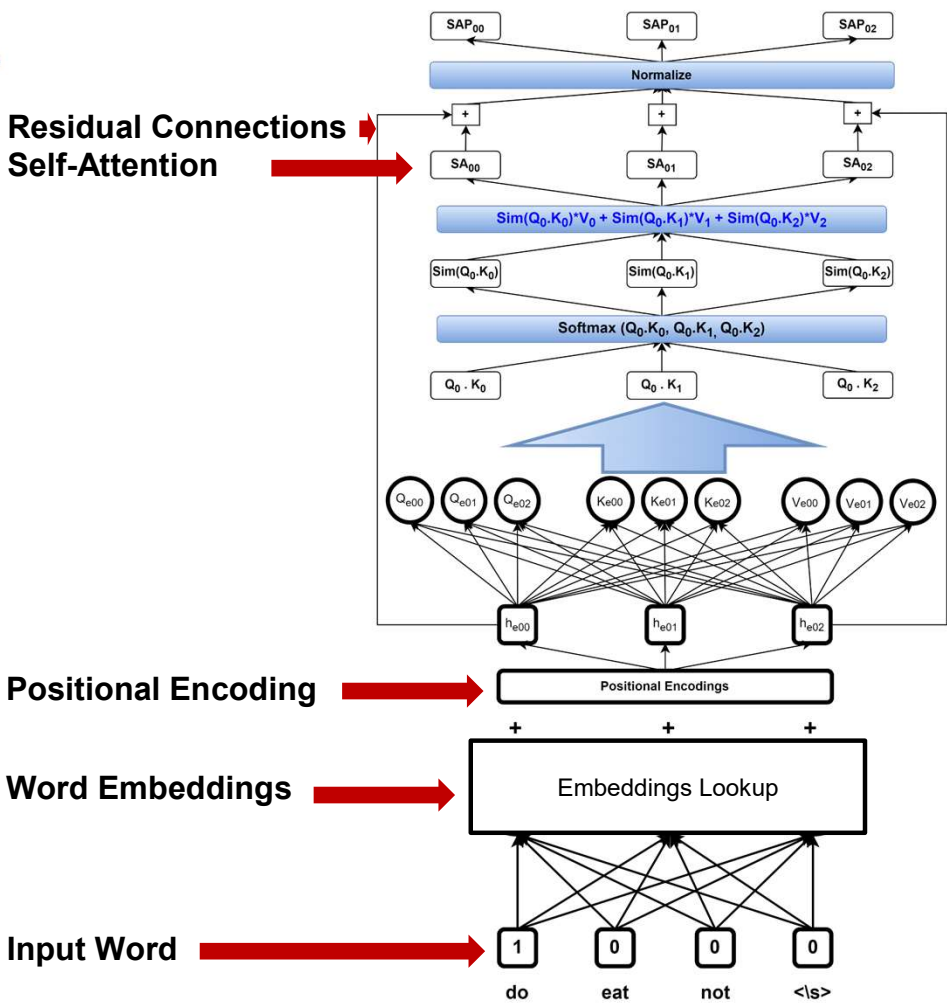
$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

- where μ and σ^2 are the mean and variance computed across the feature dimension, and γ, β are learned scale and shift parameters.

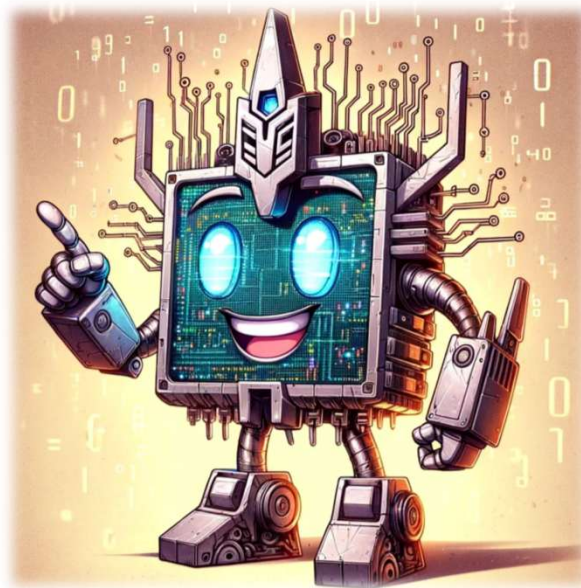
Summary: Who am I?



- So, the encoded input now has:
 - **Word information** using word embeddings
 - Words to numbers
 - **Positional Information** based on Positional Embeddings
 - Encoding order
 - **Contextual information** based on Self-Attention
 - Encoding the relationships among words
- And we get the ability to train in parallel



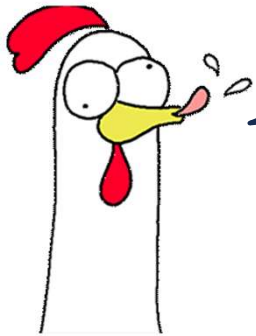
Its time to **Decode!**



Decoders

- Decoders have their **own Self-Attention** *Query, Key* and *Value* Matrices
 - As the output sentence begins, it only has itself for self-attention as we will be doing auto-regressive generation
- This self-attention will remain one-directional, as only the previous decoded states and current states have been generated at time t
 - Such one-directional behavior is implemented using “masked self-attention”.
 - Examples include **training of the decoder** where, in addition to **teacher forcing**, only the previous states are allowed to take part in the decode process, even though the future gold labels are also available
 - This also occurs in both inference and training of **decode-only transformer architectures like GPT**.
- Decoders also have a new set of *Query, Key, and Value* Matrices for **Encoder-Decoder Attention**

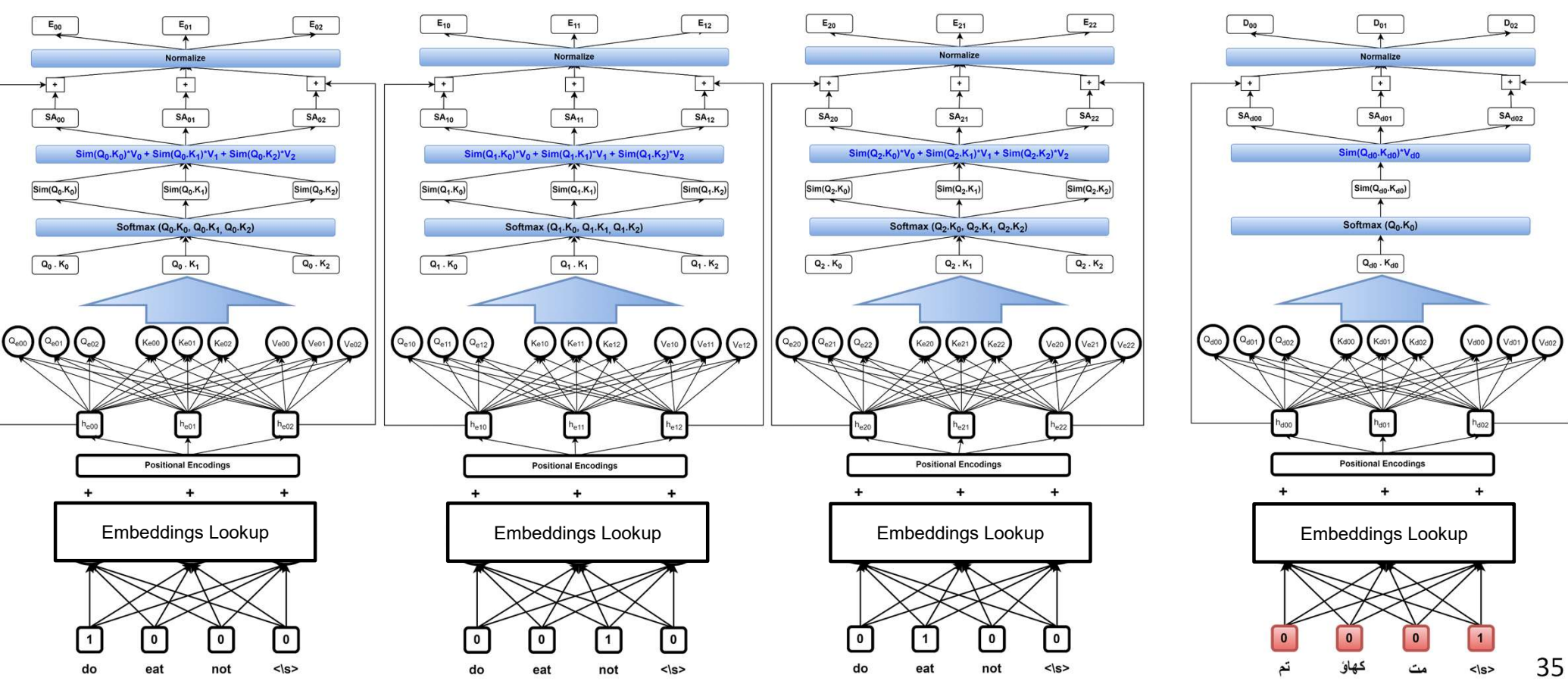
Simple... right?



Yes! Very simple!

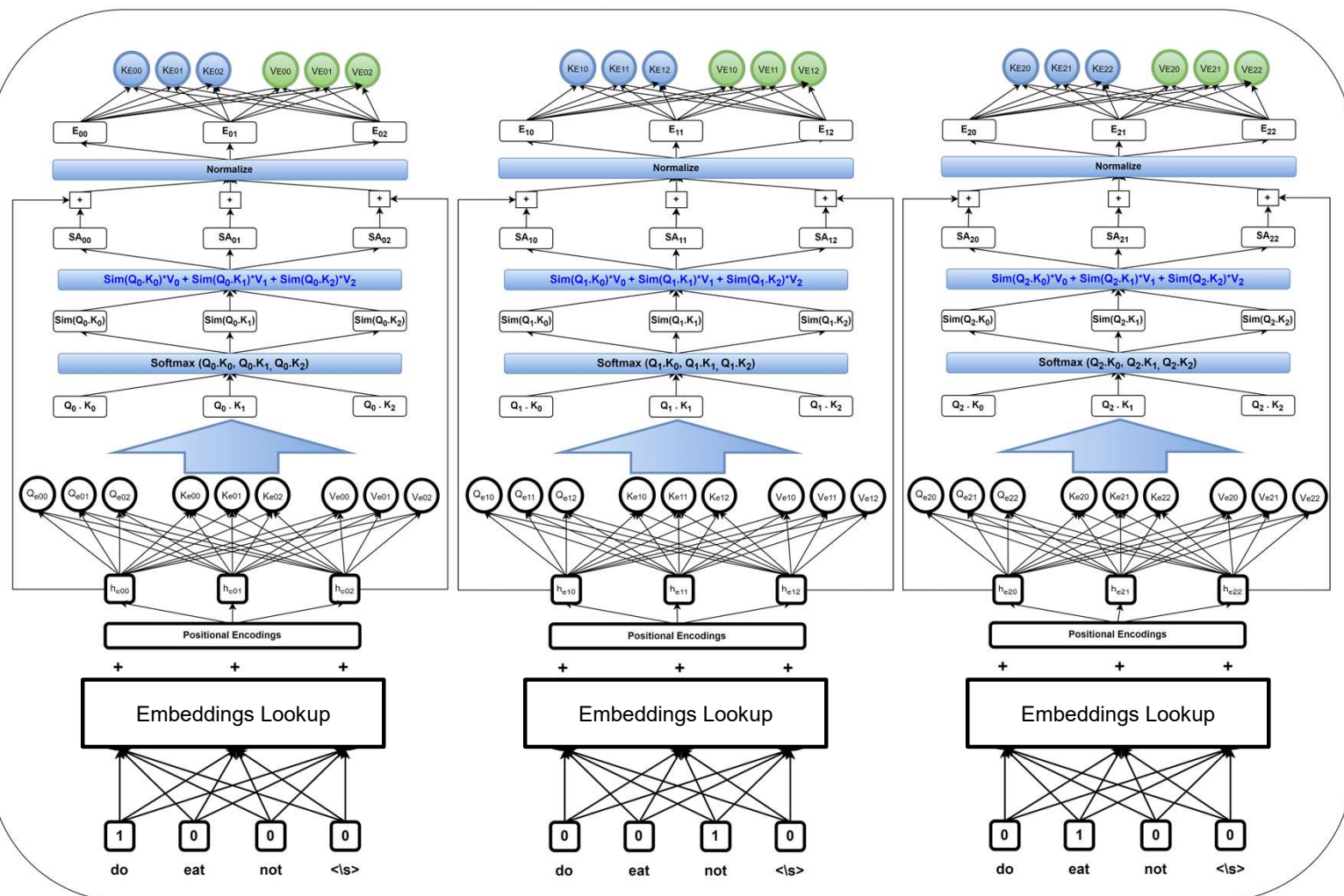
Though, I am having a hard time
recalling my name now.

And... The beginning of the End!

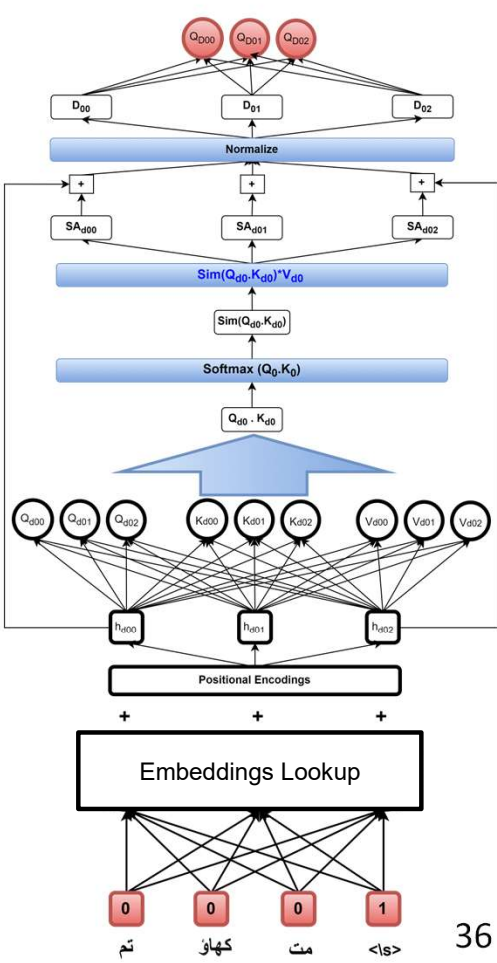


Adding in Encoder-Decoder Attention

ENCODER

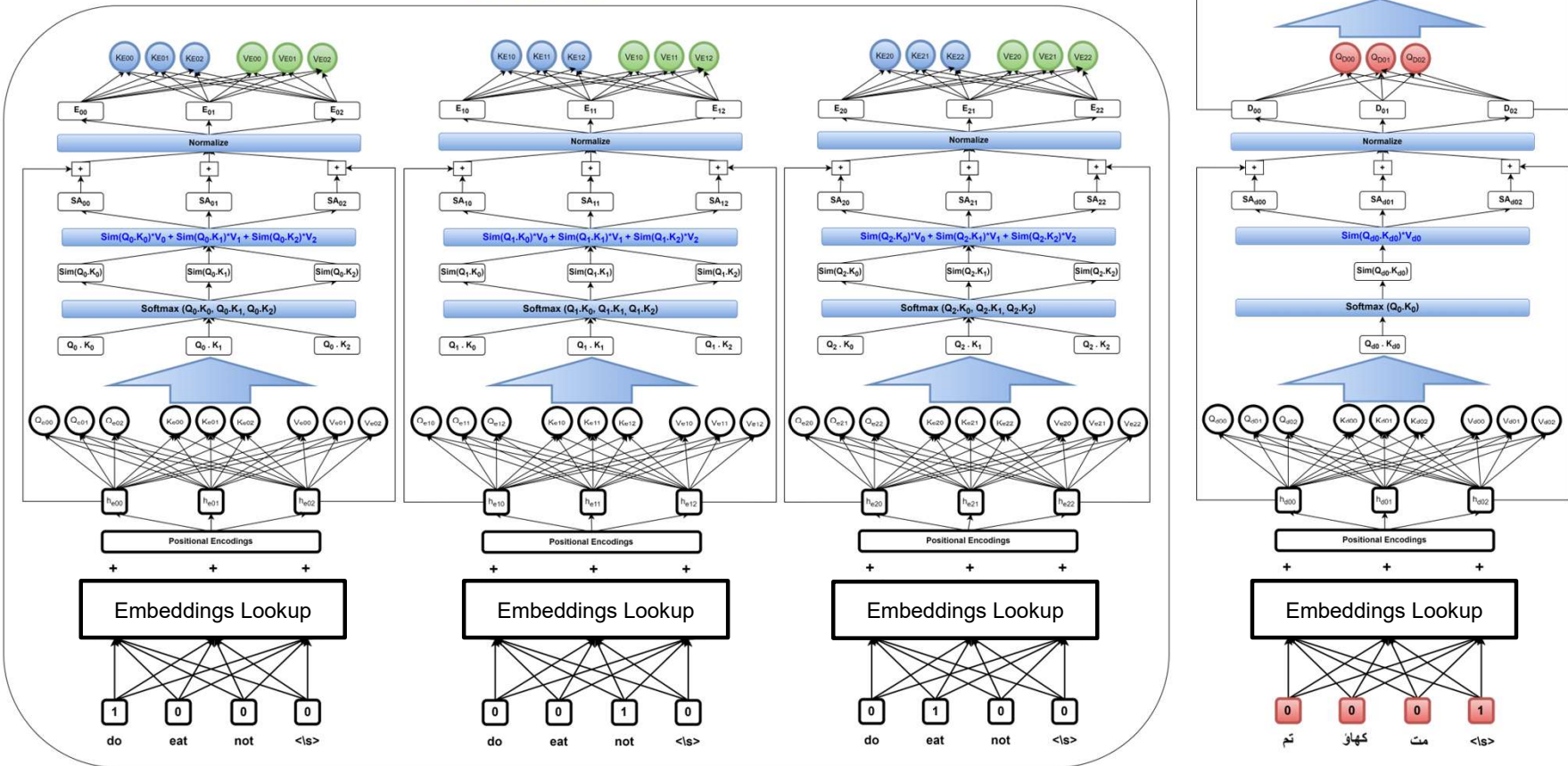


Masked Self-attention

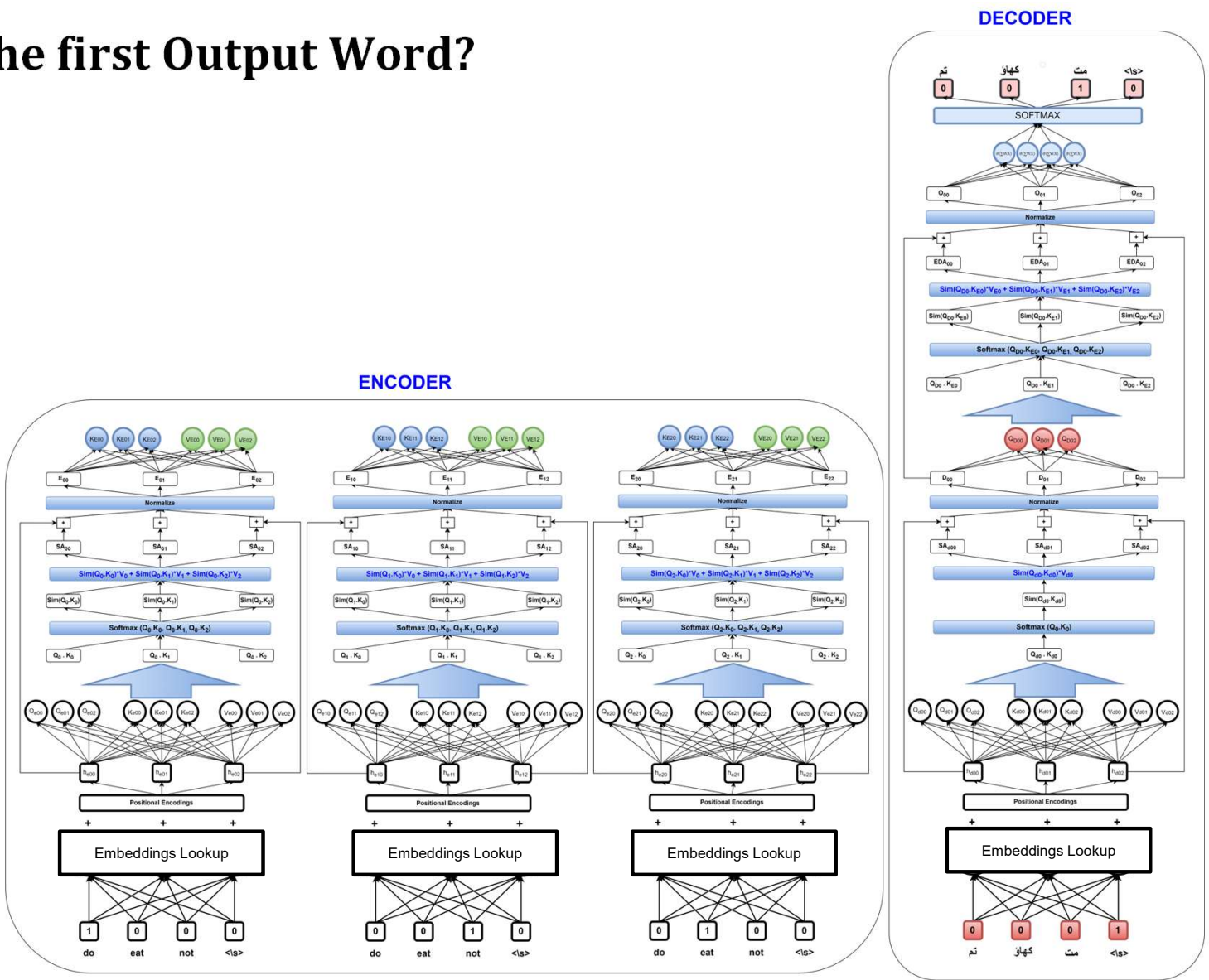


Et voilà!

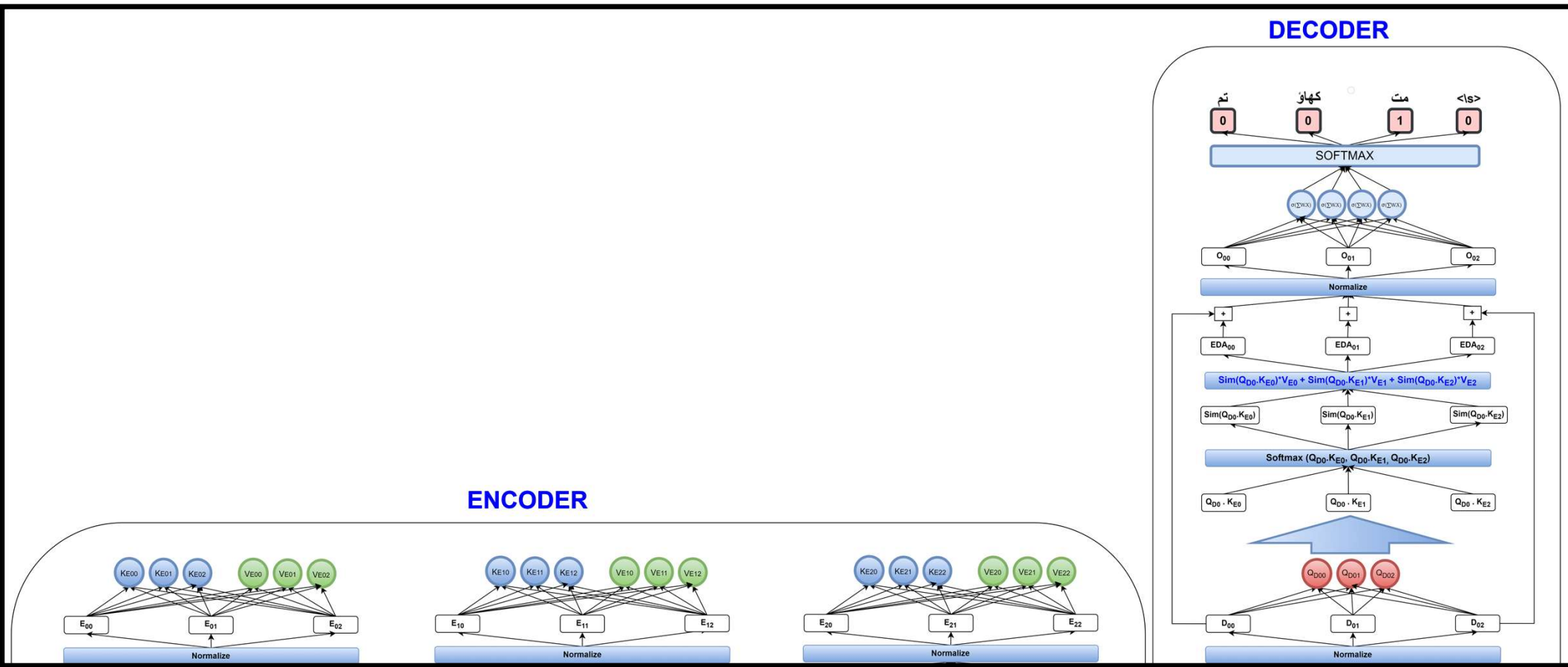
ENCODER



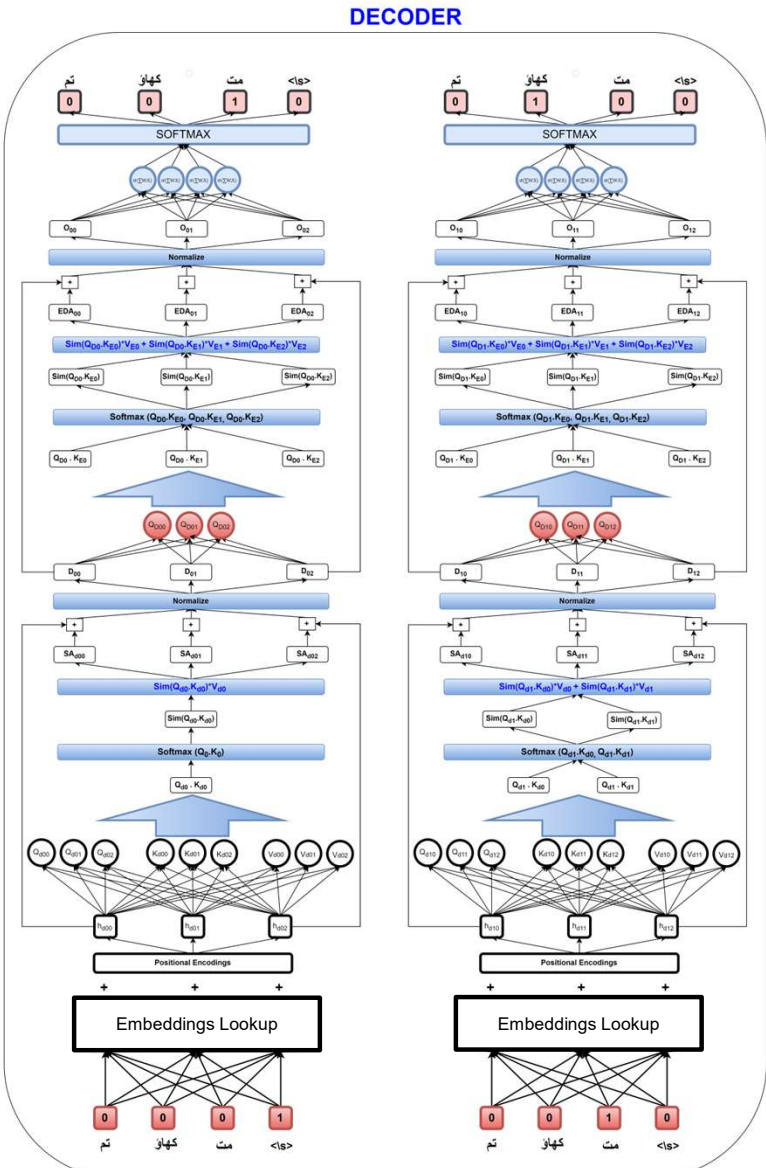
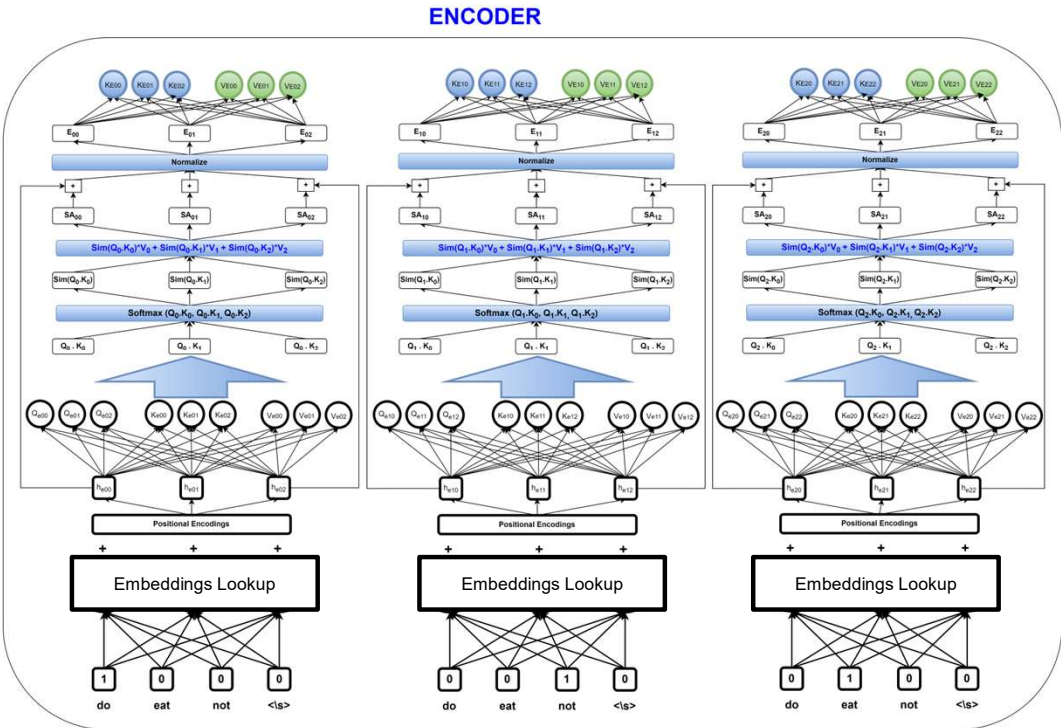
Ready for the first Output Word?



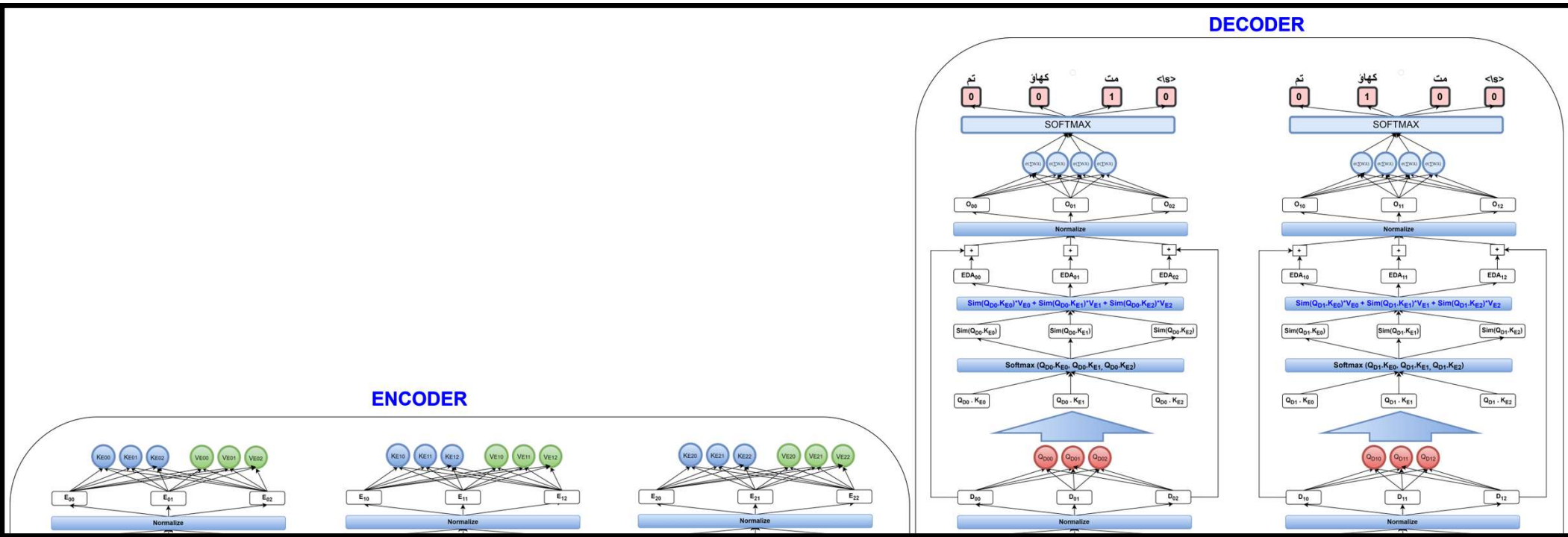
Ready for the first Output Word?



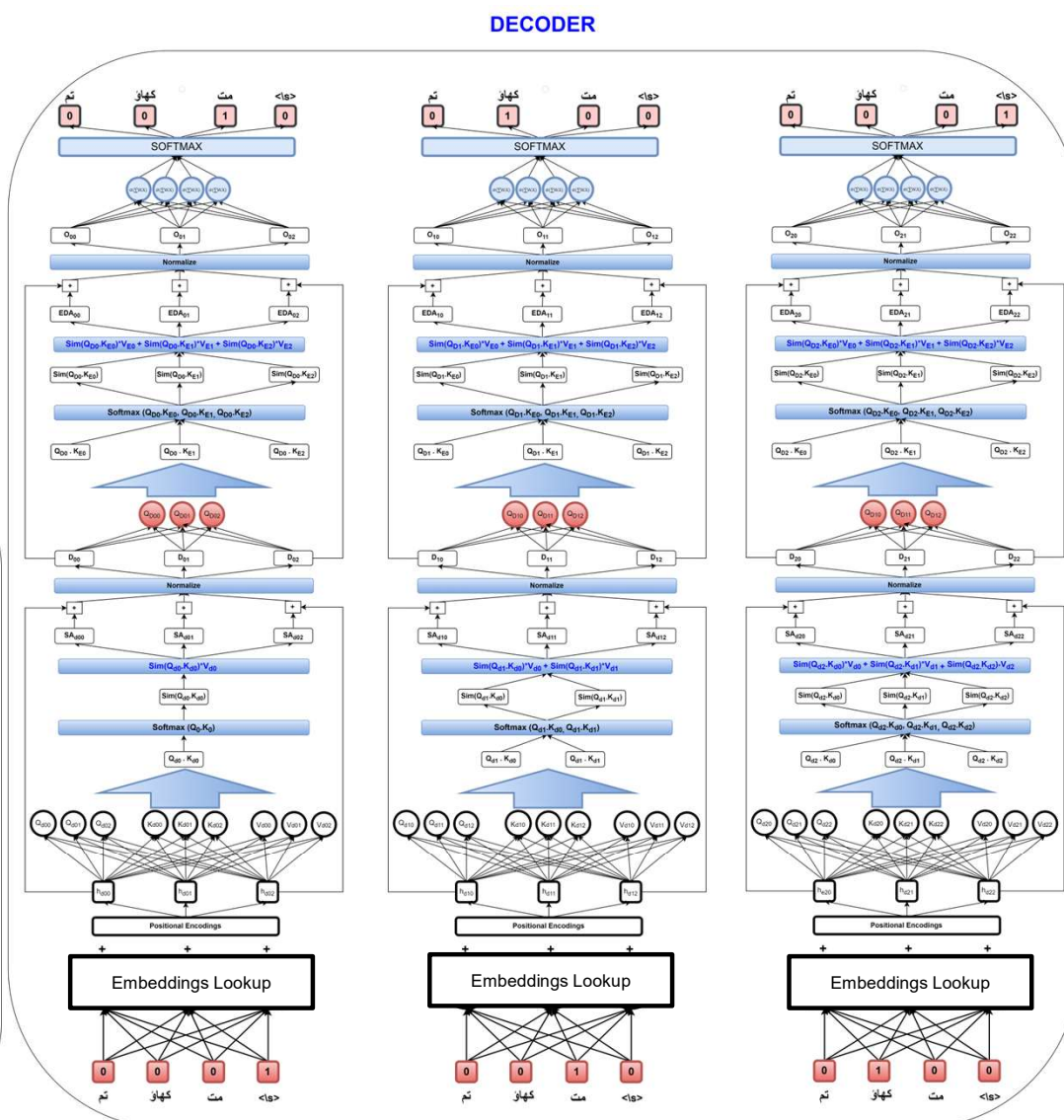
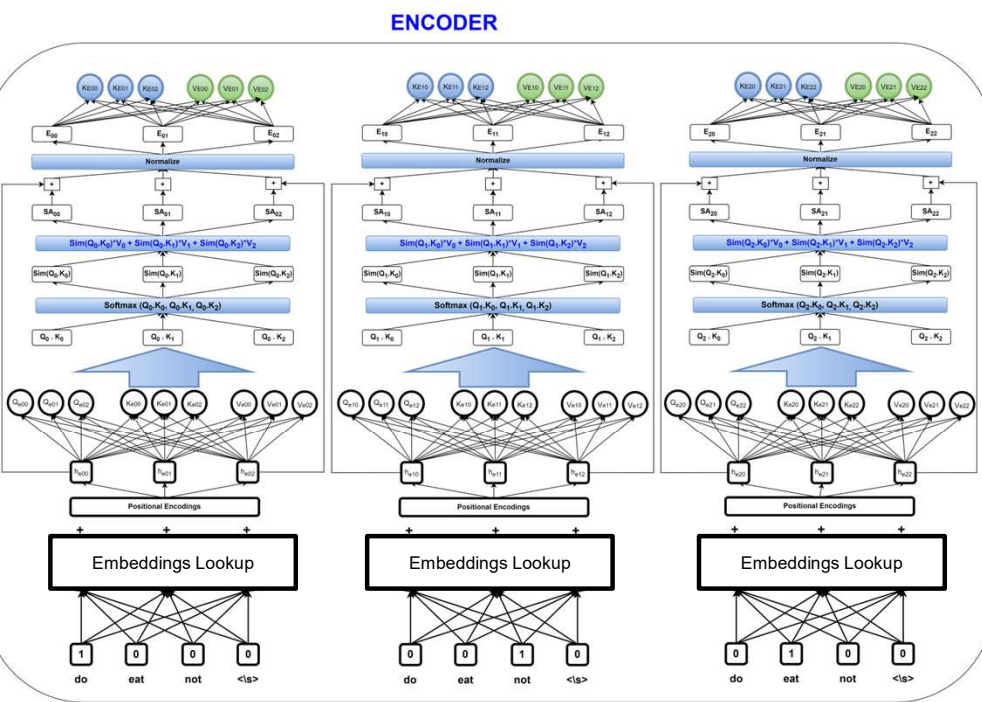
Ready for the second one?



Ready for the second one?

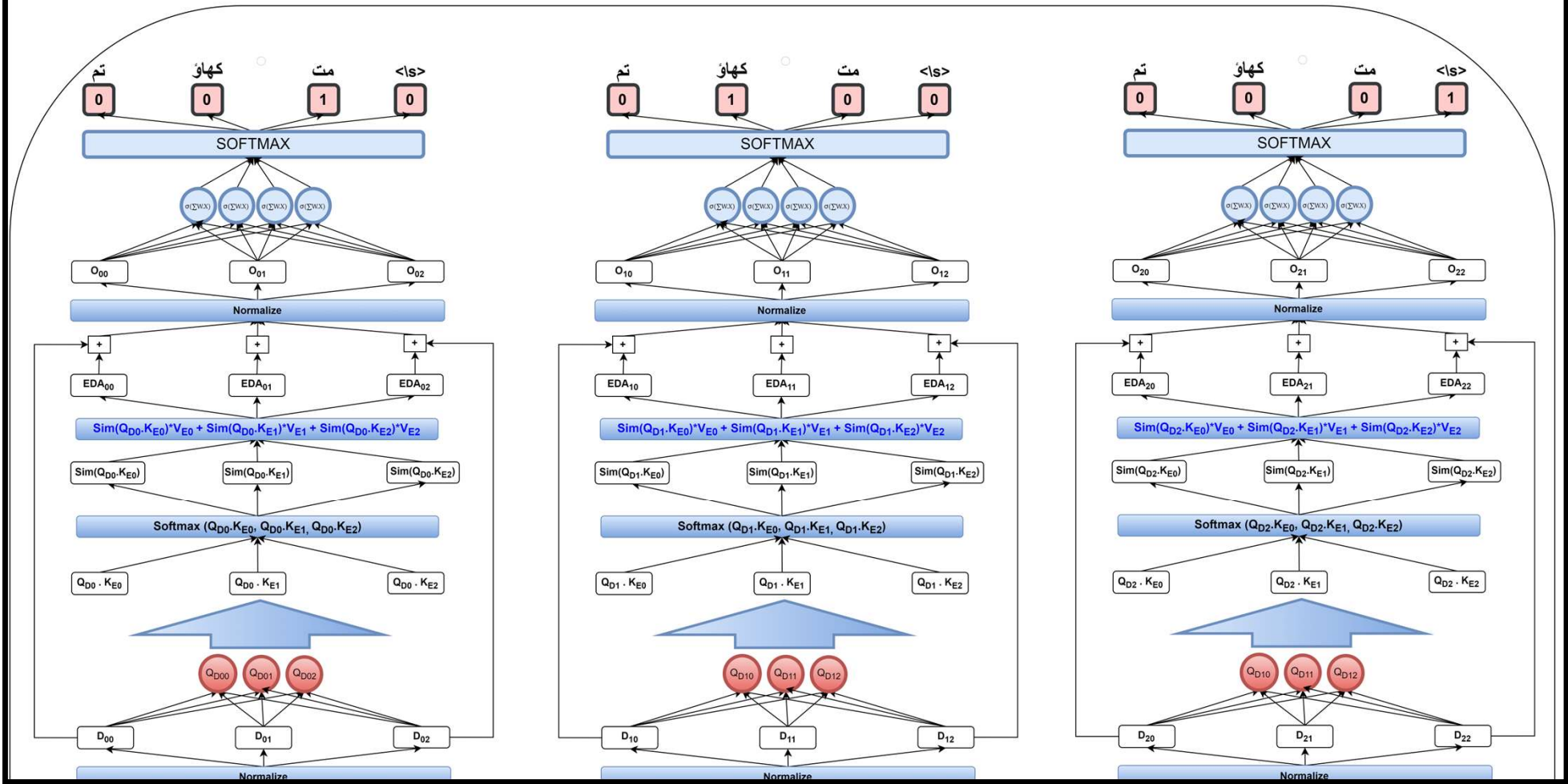


Ready for the third one?

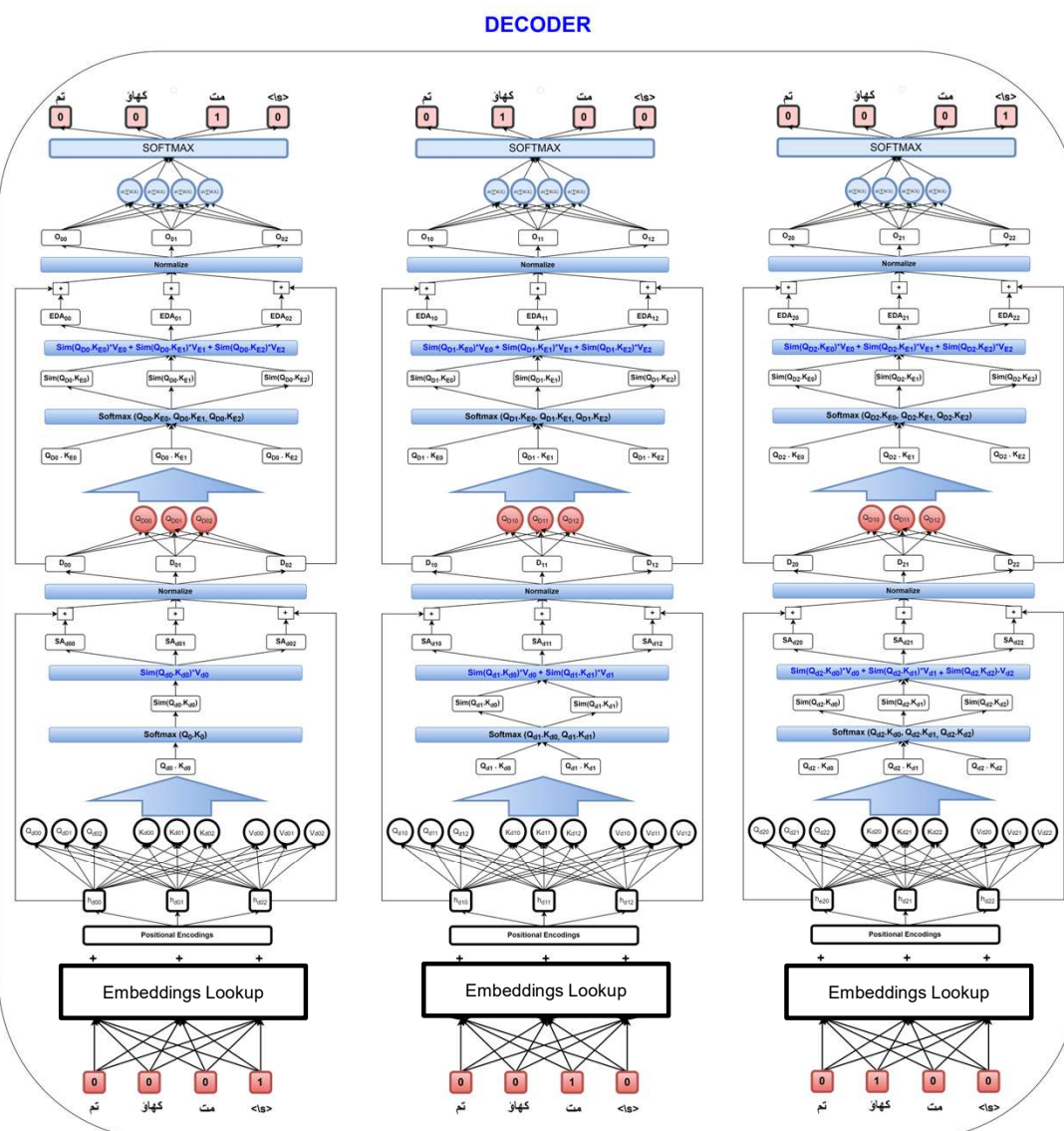
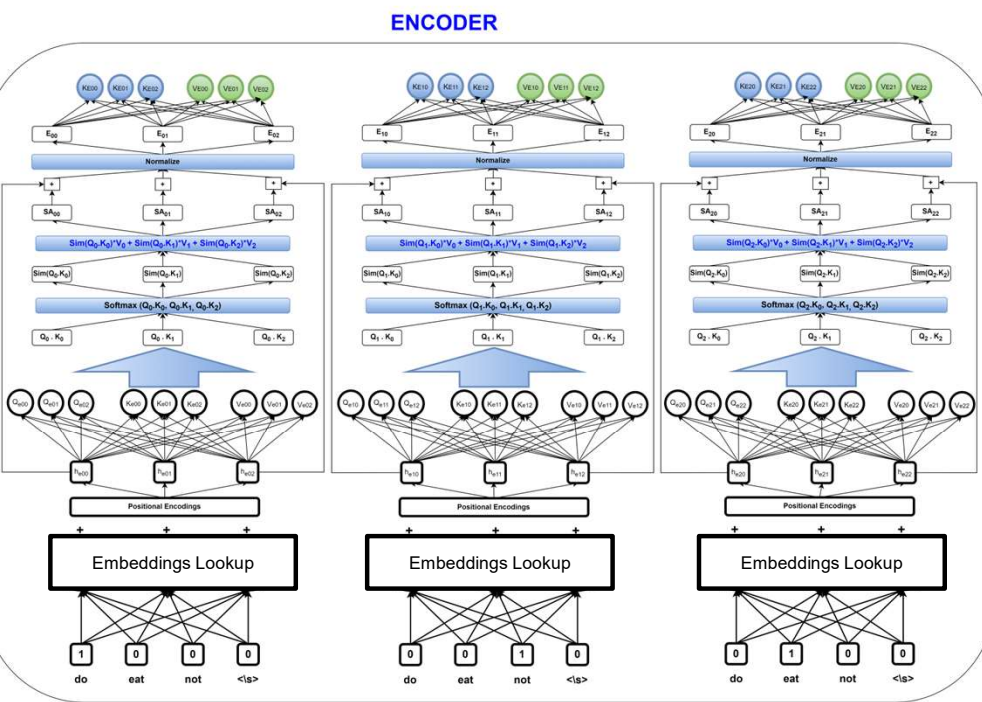


Ready for the third one?

DECODER



Ready for the third one?



Congratulations! 😊

Thanks!



Now, let's dive in mathematically!

Attention: Intuition

Imagine a reader standing in a library. Every book has a label on its spine (a **key**) and contents inside (a **value**). The reader has a question in mind (a **query**).

She walks the shelves, compares her question to each label, and then, instead of picking one book, she **takes a little of every book, more from the books whose labels match her question best**, and **blends them into a single answer**.

That is attention. Three things happen:

- **Compare** the query with every key to get a relevance score.
- **Normalize** the scores into a distribution that sums to one.
- **Blend** the values using that distribution as weights.

Attention: Intuition

Why separate keys from values?

- Because *what makes a token relevant* and *what a token contributes once chosen* are different things.
- Keys are for being found; values are for being used.

Why learn three separate projections W_Q, W_K, W_V from the same X ?

- Because the raw token vector is a general-purpose object.
- The projections let each token dress up differently for each role: one outfit for asking, one for being asked, one for answering.

Attention Applied to Tokens

Imagine reading:

“The cat sat on the mat because it was tired.”

- To understand **“it,”** we need to retrieve information about **“cat.”**
- To understand **“on,”** we need the relationship between **“sat”** and **“mat.”**

Attention lets each token gather the information it needs from other tokens.

Our story begins with how that retrieval works, then follows the problems that appear when we ask it to operate at scale.

Notation

Symbol	Meaning
n	Number of tokens in a sequence being processed
D	Model width: the number of features in each token's representation
h	Number of query heads
d	Width of each head; in our multi-head examples, $d = D/h$
$X \in \mathbb{R}^{n \times D}$	Input representations: one row per token
Q, K, V	Query, key, and value matrices
S	Attention scores, before softmax
A	Attention weights, after softmax
O	Output of an individual attention head
Y	Output after combining the heads

Step 1: Create the queries, keys, and values

Let $\mathbf{x}_i \in \mathbb{R}^{1 \times D}$ be the representation of token i . We compute:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q, \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K, \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}_V$$

where:

$$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{D \times d}$$

Each matrix is learned during training. It turns the original D features into d features useful for its particular role.

As, $d = D/h$, $d = D$ for a single head.

A projection is a collection of learned weighted sums. For example, one query feature might combine several input features to express something loosely resembling “looking for the entity being discussed.”

Those interpretations are illustrative. We do not manually assign meanings to the coordinates.

For all tokens together, and $\mathbf{X} \in \mathbb{R}^{n \times D}$:

$$\mathbf{Q} = \mathbf{X} \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X} \mathbf{W}_K, \quad \mathbf{V} = \mathbf{X} \mathbf{W}_V$$

where:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n \times d}$$

We use equal query, key, and value widths. More generally, values can have a different width; queries and keys must have matching widths for their dot product.

Step 2: Compare each query with each key

- The dot product between the query at position i and the key at position j is:

$$q_i k_j^\top = \sum_{r=1}^d q_{ir} k_{jr}$$

- r indexes a feature within the head.
- q_{ir} is feature r of query i .
- k_{jr} is feature r of key j .

The dot product adds up feature-wise agreements. A larger result means greater compatibility under the learned query and key representations.

This is a **learned compatibility score**, not necessarily ordinary semantic similarity. A pronoun can seek a noun, even though the two play different linguistic roles.

For all query–key pairs:

$$\begin{matrix} & QK^\top \\ (n \times d)(d \times n) & = n \times n \end{matrix}$$

Entry (i, j) answers:

- “How relevant is token j to the information token i is seeking?”

Scale the scores by $\sqrt{d}$

$$s = \frac{QK^T}{\sqrt{d}}$$

Why?

- A dot product adds d terms. As d increases, its typical magnitude can increase even if the individual features have not become more informative.
- We would prefer a wider head not to become more confident merely because it has more coordinates.

Scale the scores by $\sqrt{d}$

$$\mathbf{s} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}$$

Why $\sqrt{d}$?

- Suppose, as a simplifying assumption, that query and key coordinates are independent, have mean 0, and have variance 1.

Then each product $q_{ir}k_{jr}$ has:

$$\mathbb{E}[q_{ir}k_{jr}] = 0, \text{Var}(q_{ir}k_{jr}) = 1$$

Here, $\mathbb{E}$ means expected value and Var measures squared spread around the mean.

- Adding d independent terms adds their variances:

$$\text{Var}(q_i k_j^\top) = d$$

- Therefore, the standard deviation, the typical scale of the fluctuations, is: $\sqrt{d}$.
- Dividing by $\sqrt{d}$ brings that variance back to 1:

$$\text{Var}\left(\frac{q_i k_j^\top}{\sqrt{d}}\right) = \frac{d}{d} = 1$$

Scale the scores by $\sqrt{d}$

We divide by $\sqrt{d}$ because random fluctuations grow like $\sqrt{d}$, even though we add d terms.

- Dividing by d would overcorrect under these assumptions:
 - the variance would shrink to $1/d$, tending to make the scores increasingly similar
- The independence and unit-variance assumptions are an approximation, especially relevant to initialization. Training does not guarantee them exactly.
 - Training can make coordinates correlated and change variances.
 - For example, two features may begin moving together because both help identify the same linguistic relationship.
 - Random initialization and suitable scaling make the simplified picture a useful starting point, **but even at initialization, the assumptions are not guaranteed exactly**: queries and keys can come from the same input.
 - So, $\sqrt{d}$ is a sensible correction for the typical growth of dot products with dimension. **It does not guarantee that the scores always have variance exactly 1 throughout training.**

The trouble with large scores

Softmax converts scores into positive weights that sum to one:

$$A_{ij} = \frac{e^{s_{ij}}}{\sum_{r=1}^n e^{s_{ir}}}.$$

For a fixed query i , we exponentiate each score and divide by the total across the available keys. Consider:

$$\text{softmax}([1, 2, 3]) \approx [0.090, 0.245, 0.665]$$

Multiply the scores by 8:

$$\text{softmax}([8, 16, 24]) \approx [0.0000001, 0.000335, 0.999665]$$

The ranking is unchanged, but the distribution has become almost a **hard selection**.

This can also make learning through the attention weights difficult. For example:

$$\frac{\partial A_{ij}}{\partial s_{ij}} = A_{ij}(1 - A_{ij}).$$

- This derivative measures how much the attention weight changes when its score changes. It becomes small when the weight is very close to 0 or 1.
- Scaling helps prevent head width alone from pushing softmax into this saturated regime. This is also the motivation given in the original Transformer paper.

Step 4: Retrieve a weighted mixture of the values

Once the weights have been computed:

$$o_i = \sum_{j=1}^n A_{ij} v_j$$

Every value contributes, but highly attended values contribute more.

In matrix form:

$$\begin{aligned} \mathbf{O} &= \mathbf{AV} \\ (n \times n)(n \times d) &= n \times d \end{aligned}$$

Putting everything together:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right)\mathbf{V}$$

Softmax operates separately on each row.

Step 4: Retrieve a weighted mixture of the values

$$O = \textit{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V$$

Read this formula from the inside outward:

- **Compare:** compute query–key compatibility.
- **Scale:** control the numerical spread of the scores.
- **Normalize:** turn scores into attention weights.
- **Retrieve:** form a weighted sum of the values.

Numerical Example

Suppose one query attends to three tokens:

$$\mathbf{q} = [\sqrt{2}, \mathbf{0}], \quad k_1 = [1, 0], k_2 = [0, 1], k_3 = [1, 1]$$

Since $d = 2$, we divide by $\sqrt{2}$. The scores are:

$$\left[\frac{qk_1^\top}{\sqrt{2}}, \frac{qk_2^\top}{\sqrt{2}}, \frac{qk_3^\top}{\sqrt{2}} \right] = [1, 0, 1]$$

The first and third keys match equally well. Softmax gives:

$$[A_1, A_2, A_3] = \frac{[e, 1, e]}{2e + 1} \approx [0.422, 0.155, 0.422]$$

Let:

$$\mathbf{v}_1 = [2, 0], \mathbf{v}_2 = [0, 2], \mathbf{v}_3 = [2, 2]$$

Then:

$$\mathbf{o} = 0.422[2, 0] + 0.155[0, 2] + 0.422[2, 2]$$

$$\boxed{\mathbf{o} \approx [1.689, 1.155]}$$

Notice that even the token with score 0 receives some weight. **A score of zero is not an instruction to ignore a token.**

How to Ignore Tokens in Attention Calculations?

We add a mask M before softmax:

$$S = \frac{QK^T}{\sqrt{d}} + M.$$

For an allowed connection, $M_{ij} = 0$. For a forbidden connection, conceptually $M_{ij} = -\infty$.

Because:

$$e^{-\infty} = 0,$$

the forbidden token receives zero weight.

This is how we can exclude padding tokens or prevent a decoder from looking into the future.

Limitations of Single-Head Scaled Dot Product Attention

- A single head produces one attention distribution for each query. That same distribution controls how every coordinate of its value vectors is mixed.
- But a token may need several kinds of information simultaneously: an entity, a location, a syntactic relationship, and so on.
- That suggests a natural extension:
“Let the token ask several different questions, and keep the answers separate until we combine them.”

Multi-head Attention: Several Retrieval Patterns At Once

Multi-head attention, or **MHA**, runs several attention operations in parallel.

- **Each head has its own** query, key, and value projections. Consequently, each head can learn its own notion of relevance.
- In our sentence, different heads might help gather information about the cat, its location, or grammatical relationships.
 - These are possible learned behaviors, not predetermined job descriptions.
 - So, for **polysemous** or **homonymous** tokens, it may or may not learn those relations.

Polysemy and Homonymy

Polysemy and **homonymy** both involve “**one form, multiple meanings**,” but they differ in whether those meanings are **related**.

Polysemy (one word, related senses)

- A **single word** has **multiple related meanings**—you can usually feel a shared core idea.
 - **paper** = material / a newspaper / an academic article
 - **mouth** = mouth of a person / mouth of a river
 - **head** = body part / leader / top of something
- These senses are historically/semantically connected.

Homonymy (different words, same form)

- Two (or more) **different words** just **happen to look/sound the same**, but their meanings are **unrelated** (often with different historical origins).
 - **bank** = financial bank vs river bank
 - **bat** = flying mammal vs sports bat
 - **sole** = only vs fish vs bottom of a shoe (*some pairs are homonymous, some are etymologically related depending on the item*)

Quick test

- If you can explain the meanings via a **metaphor/extension** (“mouth of a river” looks like a mouth) → **polysemy**.
- If the meanings feel like a **pure coincidence** → **homonymy**.

In dictionaries, polysemous senses are typically listed under **one headword with multiple senses**, while homonyms may appear as **separate headwords** (sometimes numbered).

Multi-head Attention

Let $a \in \{1, \dots, h\}$ index the heads.

For each head:

$$Q^{(a)} = XW_Q^{(a)}, K^{(a)} = XW_K^{(a)}, V^{(a)} = XW_V^{(a)}$$

The superscript (a) identifies the head; it is not an exponent.

Each projection has shape:

$$W_Q^{(a)}, W_K^{(a)}, W_V^{(a)} \in \mathbb{R}^{D \times d}$$

The head computes:

$$O^{(a)} = \text{softmax} \left(\frac{Q^{(a)} K^{(a)\top}}{\sqrt{d}} + M \right) V^{(a)}$$

Each:

$$O^{(a)} \in \mathbb{R}^{n \times d}$$

The key addition is **a separate softmax for every head**. Each head can retrieve from different positions.

Why project down into smaller heads?

Suppose:

$$D = 512, h = 8, d = 64.$$

Every head receives the **full 512-dimensional** input and **learns its own projection** into **64 dimensions**.

A head does **not** ordinarily receive just a fixed slice of the original input features. Its projection can combine information from all 512 coordinates.

Why make each head narrower?

- Because we want multiple perspectives while keeping the total computational budget manageable.
- If eight heads each retained width 512, the combined attention representation would have width:
$$8 \times 512 = 4096$$
- That would substantially increase parameters and computation. Instead:
$$h d = D.$$
- So eight 64-dimensional heads together produce 512 features.
- The design distributes a fixed total width across several separately controlled retrieval operations.

Why does this help more than simply making one head wider?

A single wide head still produces one attention distribution per query.

For example, it might retrieve:

$$0.5 v_{\text{cat}} + 0.5 v_{\text{mat}}$$

Two heads can maintain two separately controlled retrievals:

$$o^{(1)} \approx v_{\text{cat}}^{(1)}, o^{(2)} \approx v_{\text{mat}}^{(2)}$$

The model can then decide how to combine them.

- A single head can still represent rich information.
- MHA gives it additional flexibility: **different feature groups can gather information using different attention patterns.**

Why concatenate the heads?

We join their outputs along the feature dimension:

$$\mathbf{O}_{\text{concat}} = \textit{Concat}(\mathbf{O}^{(1)}, \dots, \mathbf{O}^{(h)})$$

Therefore:

$$\mathbf{O}_{\text{concat}} \in \mathbb{R}^{n \times hd}$$

With $hd = D$:

$$\mathbf{O}_{\text{concat}} \in \mathbb{R}^{n \times D}$$

Concatenation preserves the separate answers. Averaging the heads immediately would merge them and lose the distinction between which head produced which features.

The Output Projection

We compute:

$$Y = \mathbf{O}_{\text{concat}} W_o$$

where:

$$W_o \in \mathbb{R}^{hd \times D}$$

Under our usual choice $hd = D$:

$$W_o \in \mathbb{R}^{D \times D}$$

So after concatenation, the representation already has width D .

The output projection ordinarily **mixes and transforms the combined features**; it does not increase their dimensionality.

The Output Projection

For our example:

$$8 \text{ heads} \times 64 \text{ features} = 512 \text{ features}$$

followed by:

$$512 \rightarrow 512$$

There is, however, an equivalent way to view this operation.

Split W_O into h blocks, each with shape $d \times D$. Then:

$$Y = \sum_{a=1}^h O^{(a)} W_O^{(a)}.$$

Under this view, each head's d -dimensional output is projected into the shared D -dimensional space, and those contributions are added.

Both descriptions represent the same computation:

- **Concatenate, then project.**
- Project each head into the model space, then sum.

The output projection also produces the correct width for the residual connection, which adds the attention contribution to the token's existing representation.

The Output Projection

The complete formula is:

$$\mathbf{MHA}(X) = \mathit{Concat}(\mathbf{O}^{(1)}, \dots, \mathbf{O}^{(h)}) \mathbf{W}_o$$

A useful size check:

Stage	Shape when $D = 512$, $h = 8$, $d = 64$
Input	$n \times 512$
Query, key, or value for one head	$n \times 64$
Attention weights for one head	$n \times n$
Output of one head	$n \times 64$
Concatenated outputs	$n \times 512$
Final attention output	$n \times 512$

Queries, Keys, and Values

- In Self-Attention, we extract three types of vectors/encodings from each token in the sequence. These three have different roles:
 1. The **Query** which says “Here’s what I’m interested in...”
 2. The **Key** which says “This is what I have...”
 3. The **Value** which says “If you find me interesting, here’s what I will communicate to you...”
- Can be thought of as a search engine
 - “When you search for videos on Youtube, the search engine will map your **query** (text in the search bar) against a set of **keys** (video title, description, etc.) associated with candidate videos in their database, then present you the best matched videos (**values**)”

Self-Attention: Q, K, and V

- Starting from our positionally encoded embeddings:

$$X_1, X_2, \dots, X_n, \text{ where } X_{pos} \in \mathbb{R}^{1 \times d}$$

- The three vectors are extracted by (learnable) projections:

$$k_{pos} = X_{pos} W^K \in \mathbb{R}^{1 \times d_k}, \quad W^K \in \mathbb{R}^{d \times d_k}$$

$$q_{pos} = X_{pos} W^Q \in \mathbb{R}^{1 \times d_k}, \quad W^Q \in \mathbb{R}^{d \times d_k}$$

$$v_{pos} = X_{pos} W^V \in \mathbb{R}^{1 \times d_v}, \quad W^V \in \mathbb{R}^{d \times d_v}$$

- It is helpful to keep $d_k = d_q$
- For simplicity of derivations, we will just keep $d_k = d_q = d_v$

Self-Attention: Dot-Product

- Similar to the search engine analogy, when we are looking for a good match, we focus on the **query** and the **key**
 - It only makes sense to look through *all the key vectors* when dealing with an *individual query vector* (imagine if Google only searched through one webpage for your query!)
 - Since we are dealing with vectors, we can compute a compatibility/affinity score with *dot products*

$$\begin{array}{cccccc} q_1 \cdot k_1, & q_1 \cdot k_2, & q_1 \cdot k_3, & \dots, & q_1 \cdot k_n \\ q_2 \cdot k_1, & q_2 \cdot k_2, & q_2 \cdot k_3, & \dots, & q_2 \cdot k_n \\ & & \dots & & \\ q_n \cdot k_1, & q_n \cdot k_2, & q_n \cdot k_3, & \dots, & q_n \cdot k_n \end{array}$$

Self-Attention: Vectorizing our Approach

- Before diving deeper, this would be a good point to shift from individual tokens to dealing with the sequences as a whole
 - Vectorizing the approach makes the equations much cleaner
- Instead of dealing with individual input vectors, we can concatenate all of them to get one large matrix; similarly, we could get Query, Key, and Value matrices in one clean expression

$$X = [X_1; X_2; \dots; X_n] \in \mathbb{R}^{n \times d}$$

$$X = \begin{bmatrix} - & \overrightarrow{X_1} & - \\ - & \overrightarrow{X_2} & - \\ - & \vdots & - \\ - & \overrightarrow{X_n} & - \end{bmatrix}$$

$$K = XW^K \in \mathbb{R}^{n \times d_k}, \quad W^K \in \mathbb{R}^{d \times d_k}$$

$$Q = XW^Q \in \mathbb{R}^{n \times d_k}, \quad W^Q \in \mathbb{R}^{d \times d_k}$$

$$V = XW^V \in \mathbb{R}^{n \times d_v}, \quad W^V \in \mathbb{R}^{d \times d_v}$$

Self-Attention: Vectorizing our Approach

Now that we have Query, Key, and Value matrices, we can get the affinity scores for all pairs in one go with a matrix multiplication

$$QK^T \in \mathbb{R}^{n \times n}$$

As

$$Q \in \mathbb{R}^{n \times d_k}, K \in \mathbb{R}^{n \times d_k}$$

$$Q = \begin{bmatrix} - & \vec{q_1} & - \\ - & \vec{q_2} & - \\ - & \vdots & - \\ - & \vec{q_n} & - \end{bmatrix}; K^T = \begin{bmatrix} | & | & | & | \\ \vec{k_1} & \vec{k_2} & \cdots & \vec{k_n} \\ | & | & | & | \end{bmatrix}$$

$$QK^T = \begin{bmatrix} \vec{q_1} \cdot \vec{k_1} & \vec{q_1} \cdot \vec{k_2} & \cdots & \vec{q_1} \cdot \vec{k_n} \\ \vec{q_2} \cdot \vec{k_1} & \vec{q_2} \cdot \vec{k_2} & \cdots & \vec{q_2} \cdot \vec{k_n} \\ \vdots & \vdots & \ddots & \vdots \\ \vec{q_n} \cdot \vec{k_1} & \vec{q_n} \cdot \vec{k_2} & \cdots & \vec{q_n} \cdot \vec{k_n} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Self-Attention: Scores to Weights

- Since we have (affinity) scores for each of the keys to this given query, they should be normalized to be interpreted as weights (for a weighted average later)
 - Recall for Classification, we pass the logits of a Neural Network through a Softmax to get probabilities
 - Extra step: divide the scores by the dimensionality (# embedding dimensions) of the key vector (helps with stable gradients, otherwise the Softmax input would be very high)

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}$$

Self-Attention: Scores to Weights

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) = \begin{bmatrix} \text{softmax}\left(\frac{1}{\sqrt{d_k}} \langle \vec{q}_1 \cdot \vec{k}_1, \vec{q}_1 \cdot \vec{k}_2, \dots, \vec{q}_1 \cdot \vec{k}_n \rangle\right) \\ \text{softmax}\left(\frac{1}{\sqrt{d_k}} \langle \vec{q}_2 \cdot \vec{k}_1, \vec{q}_2 \cdot \vec{k}_2, \dots, \vec{q}_2 \cdot \vec{k}_n \rangle\right) \\ \vdots \\ \text{softmax}\left(\frac{1}{\sqrt{d_k}} \langle \vec{q}_n \cdot \vec{k}_1, \vec{q}_n \cdot \vec{k}_2, \dots, \vec{q}_n \cdot \vec{k}_n \rangle\right) \end{bmatrix}$$

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) = \begin{bmatrix} s_{1,1} & s_{1,2} & \cdots & s_{1,n} \\ s_{2,1} & s_{2,2} & \cdots & s_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ s_{n,1} & s_{n,2} & \cdots & s_{n,n} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Where for each row pos ,

$$\sum_{j=0}^n s_{pos,j} = 1$$

Self-Attention: Multiply with Values

- Now with the scores normalized to a sensible range, we can begin to *aggregate* information from the other tokens into our query token, based on the scores we found
 - This is just a matrix multiplication with the Value matrix

$$H = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V \in \mathbb{R}^{n \times n} \times \mathbb{R}^{n \times d_v} = \mathbb{R}^{n \times d_v}$$

$$H = \begin{bmatrix} - & \overrightarrow{H_1} & - \\ - & \overrightarrow{H_2} & - \\ - & \vdots & - \\ - & \overrightarrow{H_n} & - \end{bmatrix}$$

We get one d_v -sized smoothie for every token X_{pos}

Self-Attention: Multiply with Values

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V = \begin{bmatrix} S_{1,1} & S_{1,2} & \cdots & S_{1,n} \\ S_{2,1} & S_{2,2} & \cdots & S_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ S_{n,1} & S_{n,2} & \cdots & S_{n,n} \end{bmatrix} \cdot \begin{bmatrix} - & \vec{v}_0 & - \\ - & \vec{v}_1 & - \\ - & \vdots & - \\ - & \vec{v}_n & - \end{bmatrix}$$

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V = \begin{bmatrix} S_{1,1} & S_{1,2} & \cdots & S_{1,n} \\ S_{2,1} & S_{2,2} & \cdots & S_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ S_{n,1} & S_{n,2} & \cdots & S_{n,n} \end{bmatrix} \cdot \begin{bmatrix} v_{1,1} & v_{1,2} & \cdots & v_{1,d_v} \\ v_{2,1} & v_{2,2} & \cdots & v_{2,d_v} \\ \vdots & \vdots & \ddots & \vdots \\ v_{n,1} & v_{n,2} & \cdots & v_{n,d_v} \end{bmatrix}$$

$S_{1,1} \cdot [v_{1,1}, v_{1,2}, \dots, v_{1,d_v}]$
+
$S_{1,2} \cdot [v_{2,1}, v_{2,2}, \dots, v_{2,d_v}]$
+
...
+
$S_{1,n} \cdot [v_{n,1}, v_{n,2}, \dots, v_{n,d_v}]$

d_v dimensional vector

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V = \begin{bmatrix} \sum_{i=1}^n S_{1,i} v_{i,1} & \sum_{i=1}^n S_{1,i} v_{i,2} & \cdots & \sum_{i=1}^n S_{1,i} v_{i,d_v} \\ \sum_{i=1}^n S_{2,i} v_{i,1} & \sum_{i=1}^n S_{2,i} v_{i,2} & \cdots & \sum_{i=1}^n S_{2,i} v_{i,d_v} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^n S_{n,i} v_{i,1} & \sum_{i=1}^n S_{n,i} v_{i,2} & \cdots & \sum_{i=1}^n S_{n,i} v_{i,d_v} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n S_{1,i} \vec{v}_i \\ \sum_{i=1}^n S_{2,i} \vec{v}_i \\ \vdots \\ \sum_{i=1}^n S_{T,i} \vec{v}_i \end{bmatrix} \in \mathbb{R}^{n \times d_v}$$

Self-Attention: Multiply with Values

- The Attention mechanism results in a series of weighted averages of the rows of V , where the weighting depends on the input queries and keys.
- Each of the n queries in Q results in a specific weighted sum of the value vectors.
- Of note is that there are no learnable parameters in this particular procedure. It is entirely comprised of matrix and vector operations.

Multiheaded Attention

- Multi-Head Attention is an extension of Scaled Dot-Product Attention, in which linear transformations are run on the queries, keys, and values to yield multiple sets of inputs to the attention function.
- The attention function is computed in parallel given each of these sets of inputs, and the results are concatenated side-by-side into one matrix.
- The final result is obtained via a linear transformation of the concatenated matrix to obtain a matrix with the desired dimensionality.

$$\mathbf{Multihead}(Q, K, V) = \mathbf{Concat}(\mathbf{head}_1, \mathbf{head}_2, \dots, \mathbf{head}_h) \mathbf{W}^O \in \mathbb{R}^{n \times h \cdot d_v} \times \mathbb{R}^{h \cdot d_v \times d} = \mathbb{R}^{n \times d}$$

- Each head i is the result of running Scaled Dot-Product Attention on the i th set of transformed queries, keys, and values:

$$\mathbf{head}_i = \mathbf{Attention}(\mathbf{XW}_i^Q, \mathbf{XW}_i^K, \mathbf{XW}_i^V)$$

- Where

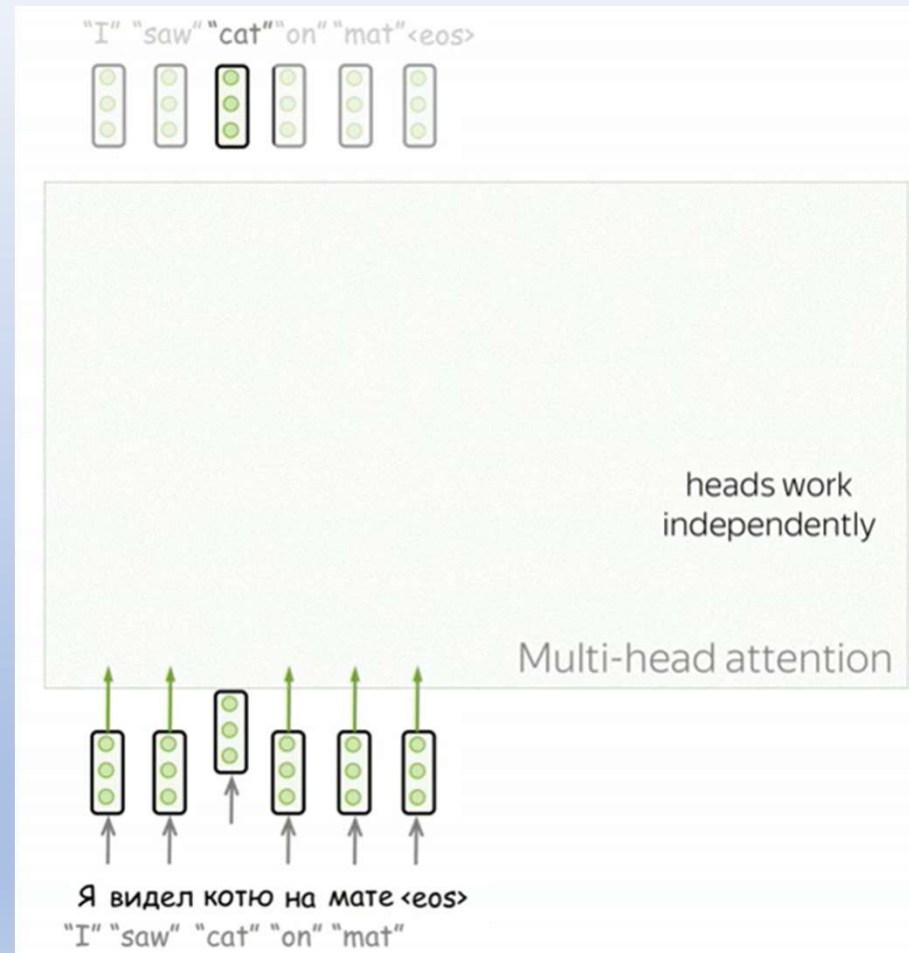
$$\mathbf{X} \in \mathbb{R}^{n \times d}$$

- Further, given a hyperparameter h indicating the number of attention heads:

$$\mathbf{W}_i^Q \in \mathbb{R}^{d \times d_k}, \quad \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}, \quad \mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}, \quad \mathbf{W}^O \in \mathbb{R}^{h \cdot d_v \times d}, \quad \mathbf{head}_i \in \mathbb{R}^{n \times d_v}$$

- $\mathbf{head}_i$ contains d_v - sized vectors for all n tokens.

Multi-Head Attention: Independently Focus on Different Things



Notes: Multiheaded Attention – An Intuitive Review

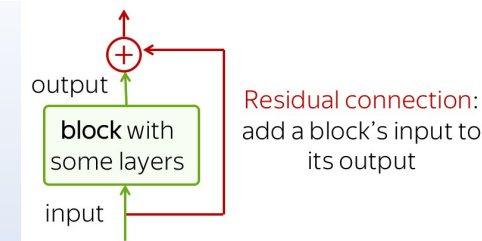
- Let's perform a quick sanity check that the matrix multiplication works out.
- First, we know that:
 - Each matrix $head_i$ will have the same number of rows as QW_i^Q , and the same number of columns as VW_i^V .
 - Since $QW_i^Q \in \mathbb{R}^{n \times d_k}$ and $VW_i^V \in \mathbb{R}^{n \times d_v}$ this means $head_i \in \mathbb{R}^{n \times d_v}$.
 - When we concatenate h of these $head_i$ together, we have a matrix in $\mathbb{R}^{n \times h \cdot d_v}$
 - **Multiplying with $W^O \in \mathbb{R}^{h \cdot d_v \times d}$ yields a matrix in $\mathbb{R}^{n \times d}$.**
 - This makes sense - we started with n queries in Q , and we ended up with n responses in the output of the Multi-Head operator.
- Notice how each head computation has a different linear transformation for the key, query, and value matrices. Each of these transformations is learned during training.

Multiheaded Attention – Applications

Multi-head attention is used in three ways in *Attention*:

1. Computing **self-attention in the encoder blocks**,
 - We have already seen the encoder's bi-directional self-attention.
2. Computing **self-attention with masking in the decoder blocks**, and
 - The decoder uses masked self-attention due to autoregressive generation
 - i.e., it only has access to previous states.
3. **Encoder-decoder attention** within the decoder blocks.
 - The encoder-decoder self-attention uses Q from the decoder while K and V from the encoder.

Residual Connections



- To stabilize and speed up training, each sub-layer in the encoder (multi-head attention and feed-forward network) is followed by a residual connection and layer normalization.
- Residual connections help mitigate the vanishing gradient problem and allow for the training of deeper networks.
- The basic idea is to add the input of the sub-layer to its output. Mathematically, it can be expressed as:

$$\textit{Output} = \textit{LayerNorm}(x + \textit{subLayer}(x))$$

- Where
 - x is the input to the sub-layer (which could be either the multi-head attention mechanism or the feed-forward network).
 - **Sublayer(x)** is the output from the sub-layer after processing x . This could be the output from either the multi-head attention mechanism or the feed-forward network.
 - $x + \textbf{Sublayer}(x)$ is the residual connection, which adds the original input x to the sub-layer's output. This addition helps the gradients flow more easily through the network, preventing them from becoming too small (vanishing gradient problem) or too large (exploding gradient problem).